

# **CHAPTER 1**

## **Introduction**

### **1.1 Background and Motivation**

Supply chain management (SCM) involves the coordination and management of all activities involved in the production and delivery of goods and services from suppliers to end-users. SCM is a complex process that involves multiple stakeholders, including suppliers, manufacturers, distributors, and retailers. The traditional approach to SCM involves a centralized system, where information is stored in a single location and accessed by all stakeholders. However, this approach can be inefficient, prone to errors, and lacks transparency.

Blockchain technology provides a decentralized, secure, and transparent system for tracking products and transactions. Blockchain is a digital ledger that records transactions in a secure and immutable manner. Transactions are verified by a network of nodes, and once verified, they are added to the blockchain. The blockchain is distributed across multiple nodes, which means that it is virtually impossible to hack or tamper with.

Information technology revolutionized logistics and supply chain and its contribution is substantial. Supply chain is one of the most important application areas where blockchain has the greatest potential to reshape the supply chain structure. Integrating blockchain in supply chain management can improve the process of the supply chain management. Blockchain is a technology with unique combination of features such as decentralized structure, distributed nodes and storage mechanism, consensus algorithm, smart contracting, and asymmetric encryption to ensure network security, transparency, and visibility.

### **1.2 Problem Statement**

The traditional approach to SCM has several challenges that need to be addressed. These challenges include issues of transparency, traceability, efficiency, and security. The lack of transparency in traditional SCM makes it difficult for stakeholders to track products as

they move through the supply chain. This lack of traceability can lead to delays, errors, and even fraud. In addition, the centralized nature of traditional SCM makes it vulnerable to hacking and tampering, which can compromise the security of the supply chain. This report will explore the potential benefits and challenges of integrating blockchain technology in supply chain management, with a focus on its impact on transparency, efficiency, and sustainability.

### **1.3 Objectives of the Study**

The primary objective of this study is to design and implement a decentralized, transparent, and secure Supply Chain Management System using Ethereum blockchain technology. The project aims to bridge the gap between traditional centralized logistics and modern distributed ledgers to ensure product authenticity and real-time traceability.

The specific objectives of this study are categorized as follows:

#### **A: Technical Objectives**

**Immutable Record Keeping:** To implement a Solidity-based smart contract on the Sepolia Testnet that records every transaction and status change, ensuring that data cannot be tampered with or deleted once it is committed to the blockchain.

**Real-time Event Synchronization:** To develop a backend ‘Watcher’ service using Node.js and Ethers.js that listens to on-chain events and automatically synchronizes a MySQL database, providing high-speed data access without compromising the source of truth.

**Role-Based Access Control (RBAC):** To design an authentication and authorization system that provides customized dashboards for seven distinct stakeholders: Admin, Supplier, Manufacturer, Manager, Warehouse Officer, Distributor, and Retailer.

#### **B. Functional Objectives**

**End-to-End Traceability:** To track the lifecycle of a product from the initial ‘Raw Material Request’ by the Supplier to the ‘Final Receipt Verification’ by the Retailer, providing a visual ‘live pipeline’ for tracking.

**Automated Bill of Materials (BoM) Management:** To automate the calculation and request of raw materials required for production based on predefined product formulas stored in the system. **Inventory Integrity:** To maintain an accurate audit of stock levels across the Warehouse and Distributor hubs, ensuring that inventory is automatically adjusted as items move through the supply chain phases.

### **C. Strategic Objectives**

**Transparency and Accountability:** To provide a verifiable history for every order, allowing any participant in the chain to audit the actor (wallet address), role, and timestamp of every milestone.

**Reduced Intermediary Dependency:** To minimize manual paperwork and human error by utilizing a single ‘Shared Ledger’ that all participants can trust simultaneously.

**Operational Efficiency:** To identify bottlenecks in the supply chain by analyzing the time taken between different statuses (e.g., from Manufacturing to Warehouse Dispatch).

## **1.4 Scope and Significance**

The traditional approach to SCM has several challenges that need to be addressed. These challenges include issues of transparency, traceability, efficiency, and security. The lack of transparency in traditional SCM makes it difficult for stakeholders to track products as they move through the supply chain. This lack of traceability can lead to delays, errors, and even fraud. In addition, the centralized nature of traditional SCM makes it vulnerable to hacking and tampering, which can compromise the security of the supply chain. This report will explore the potential benefits and challenges of integrating blockchain technology in supply chain management, with a focus on its impact on transparency, efficiency, and sustainability.

## **CHAPTER 2**

### **Literature Review**

#### **2.1 Overview of Supply Chain Management**

Supply chain management is an important aspect of the business world as it involves the coordination and management of various activities involved in the manufacturing, sourcing, distribution, and delivery of goods and services to the final consumer. It is the process that links suppliers, manufacturers, distributors, wholesalers, retailers, and other intermediaries to ensure that products reach their intended destination timely, cost-effectively, and efficiently. This essay will explore the importance of supply chain management, its components, challenges, and strategies for its effective implementation. Implementing effective supply chain management requires businesses to adopt several strategies, including collaboration, risk management, technology adoption, and continuous improvement. Collaboration involves building strong relationships with suppliers, customers, and other stakeholders through open communication and shared goals. Risk management involves identifying potential risks and developing strategies to minimize their impact. Technology adoption involves investing in tools and software that enable efficient tracking and monitoring of supply chain activities. Continuous improvement involves regularly evaluating and improving supply chain processes to enhance efficiency and effectiveness. Companies use Radio Frequency Identification (RFID), Barcode scanning, Global Positioning system, ERP systems, Transportation management system, Cloud based solutions etc., But integrating blockchain technology provides a secure and transparent way for companies. Blockchain creates a digital record of transactions, which cannot be altered or tampered.

#### **2.2 Challenges in Supply Chain Management**

Various challenges in supply chain management, including the lack of visibility, forecasting demand, managing inventory, managing suppliers, transportation management, the use of technology, sustainability, and international trade. One of the biggest challenges in supply chain management is the lack of visibility at various stages of the supply chain. It is often difficult to track the movement of raw materials, inventory,

and finished products. Accurately predicting customer demand is essential to optimize the supply chain. However, forecasting demand can be challenging because of changing market trends and consumer behaviour. Maintaining optimal inventory levels is crucial to avoid stockouts or excess inventory. However, managing inventory can be a challenge due to the impact of factors such as lead times, demand variability, and supplier reliability. Managing suppliers is essential to ensure timely delivery of materials and products. It can be a challenge to build and maintain strong relationships with suppliers, especially when dealing with a vast number of suppliers. Efficient transportation is essential to ensure timely delivery of products. However, managing transportation can be a challenge due to issues such as carrier capacity, fuel prices, and regulatory compliance. The increasing use of technology in supply chain management has created new challenges, including the implementation and integration of complex systems, data management, and cybersecurity. Consumers are demanding more sustainable products and practices, which are transforming supply chain management. However, implementing sustainable practices can be challenging due to issues such as cost, complexity, and lack of resources. The growth of international trade has made supply chain management more complex, with challenges such as language barriers, different regulatory requirements, and cultural differences.

## **2.3 Blockchain Technology**

Blockchain technology has evolved significantly since its introduction in 2008 with Bitcoin, and today it is being widely explored in supply chain management. In this domain, blockchain enables secure, transparent, and tamper-proof tracking of goods as they move through different stages—from raw material sourcing to final delivery. Each transaction or movement of goods is recorded as a block, creating a permanent and traceable history that all participants in the supply chain can access.

Just like in cryptocurrency systems, blockchain in supply chains operates as a decentralized digital ledger. Every block contains verified data such as product origin, shipment details, timestamps, and ownership transfers. These blocks are linked using cryptographic hashes, ensuring that once data is recorded, it cannot be altered without

affecting the entire chain. This increases trust among stakeholders like manufacturers, suppliers, distributors, and retailers.

The introduction of Ethereum in 2015 further enhanced blockchain capabilities by enabling smart contracts. In supply chains, smart contracts can automate processes such as payments, order fulfillment, and compliance verification when predefined conditions are met. For example, a payment can be automatically released once goods are delivered and verified.

Blockchain also helps solve common supply chain issues like lack of transparency, fraud, counterfeit products, and inefficiencies. By providing real-time visibility and a single source of truth, it improves coordination and accountability across the network. However, challenges such as scalability, integration with existing systems, and user adoption still remain.

Overall, blockchain has strong potential to revolutionize supply chain management by making it more efficient, secure, and transparent, while reducing costs and increasing trust among all participants.

## **2.4 Main Features of Blockchain Management**

- **Decentralized:** Blockchain technology is decentralized and distributed across a network of computers, which means that it is not controlled by any one central authority, government, or individual.
- **Transparency:** All transactions on the blockchain are visible to everyone on the network, and once they are recorded, they cannot be altered.
- **Security:** Blockchain technology uses cryptographic algorithms to secure data, so it is virtually impossible to hack or alter the data stored on a blockchain without being detected.
- **Immutability:** Once data is recorded on a blockchain, it cannot be deleted, edited, or manipulated, making it an ideal technology for storing permanent, unalterable records.

- **Efficiency:** Blockchain technology enables faster, more efficient transactions because it eliminates the need for intermediaries and central authorities to verify and authorize transactions.
- **Smart Contracts:** Smart contracts are self-executing agreements that automatically enforce the terms of the contract when certain conditions are met, which makes them ideal for automating complex business processes.
- **Privacy:** Blockchain technology can be used to create private or permissioned networks, which allow only authorized parties to access and view the data stored on the blockchain.

## 2.5 How Blockchain technology works?

The process begins with a user initiating a transaction, which is then verified by a network of nodes using complex algorithms to ensure its validity. This verified transaction is then added to a block, along with other verified transactions, forming a chain of blocks hence the name 'blockchain'. Each block contains a unique code called a hash that is generated based on the information stored within the block. The hash of each block is linked to the hash of the previous block, creating a chain of blocks that are chronologically connected to one another. Once a block is added to the chain, it cannot be altered or deleted without altering or deleting all the subsequent blocks in the chain (due to the cryptographic link between the blocks). This decentralized system ensures that there is no single point of failure or hacking vulnerability, makes the system more secure, transparent and immutable. The transparency of blockchain makes it possible to track transactions in real-time, and the immutability of the data prevents any tampering with the information stored on the blockchain. This technology has the potential to revolutionize industries like finance, supply chain, healthcare, and many others, improving the way we manage and store data.

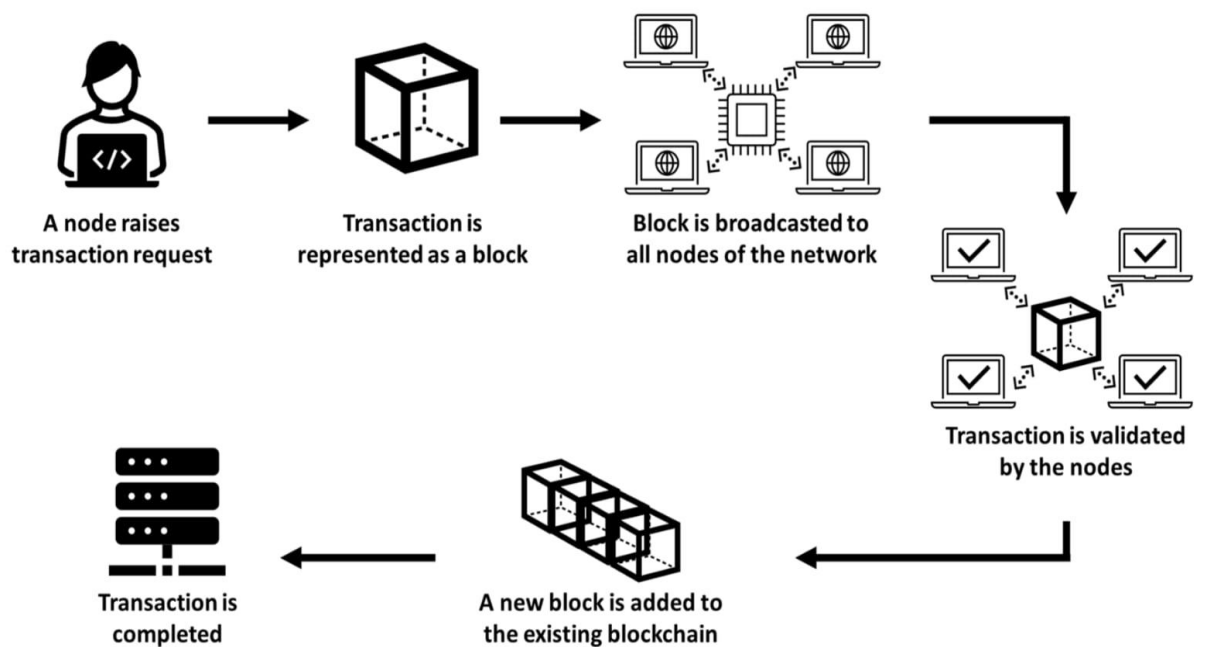
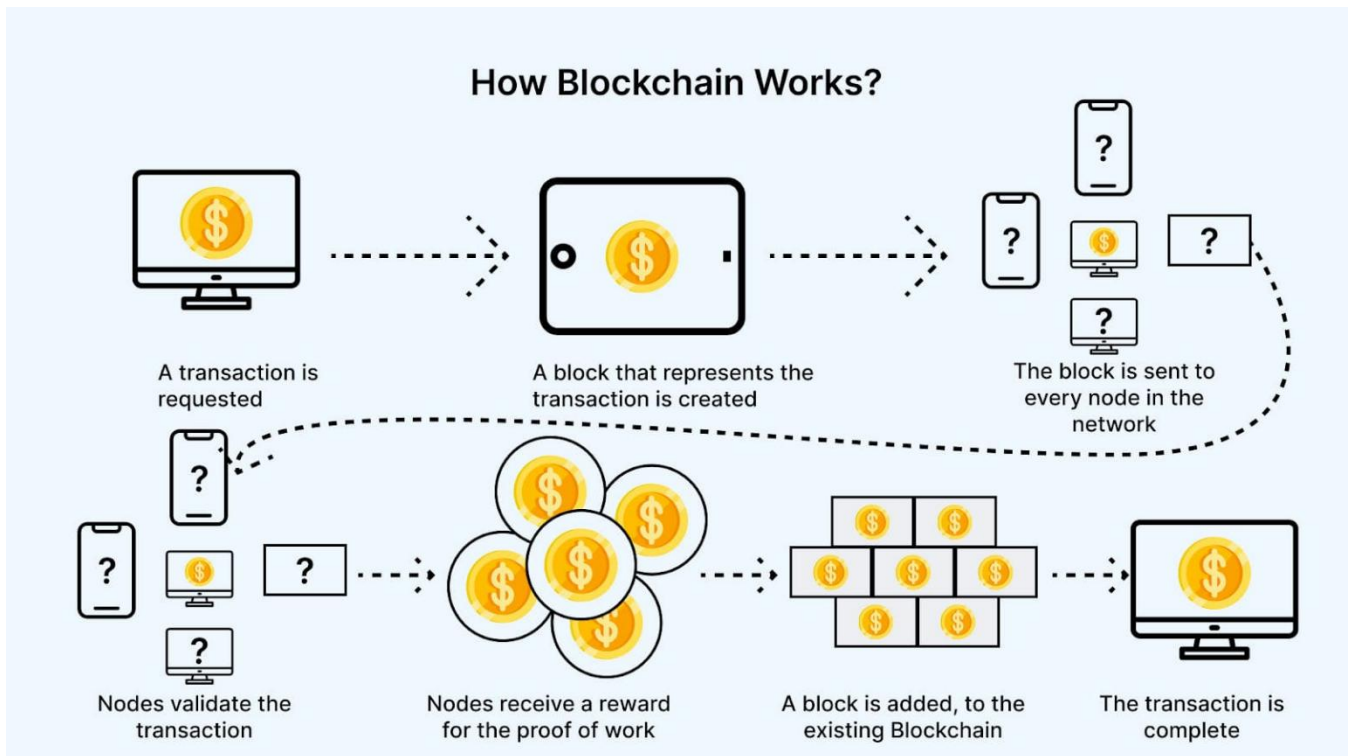


Figure 1: How the blockchain technology works.



## 2.6 Current Use Cases of Blockchain in Supply Chain Management

There are several current use cases of blockchain in SCM, including:

**Food traceability:** Blockchain can be used to trace the movement of food products through the supply chain, providing consumers with information about the origin and quality of their food.

**Pharmaceuticals:** Blockchain can be used to track the movement of pharmaceutical products through the supply chain, ensuring their authenticity and preventing the distribution of counterfeit products

**Luxury goods:** Blockchain can be used to track the movement of luxury goods, such as high-end watches and jewellery, through the supply chain, providing consumers with assurance of their authenticity.

**Energy management:** Blockchain can be used to manage energy production and distribution, providing a transparent and secure way to track the movement of energy through the supply chain.

**Automotive industry:** Blockchain can be used to track the movement of automotive parts through the supply chain, ensuring their authenticity and reducing the risk of counterfeit products.

## 2.7 Summary of Literature Review

The literature review explored the concept of supply chain management (SCM) and the role of blockchain technology in transforming SCM. Traditional SCM involves a centralized system that is prone to errors, lacks transparency, and is inefficient. Blockchain technology provides a decentralized, secure, and transparent system for tracking products and transactions. This makes it an attractive solution for supply chain management.

The literature review also discussed the benefits of integrating blockchain in SCM, including increased transparency, traceability, and efficiency, as well as improved security and reduced costs. However, there are also challenges associated with integrating blockchain in SCM, such as interoperability issues, regulatory and legal barriers, and the need for technical expertise. Current use cases of blockchain in SCM were also examined, including the example of Walmart's blockchain-based system for tracking food products.

The study found that the use of blockchain in SCM has the potential to transform the industry by increasing efficiency, reducing costs, and improving transparency and security. The comparison between blockchain and traditional SCM showed that blockchain provides a more secure, transparent, and efficient system for tracking products and transactions.

However, the adoption of blockchain in SCM still faces challenges that need to be addressed in order to fully realize its potential. The literature review highlights the potential benefits of blockchain technology in transforming the SCM industry and improving supply chain efficiency and transparency. However, further research is needed to address the challenges associated with blockchain adoption in SCM and to identify the best practices for integrating blockchain into supply chain management.

## CHAPTER 3

### System Architecture and Design

#### 3.1 Overview of Proposed System

The proposed system is a comprehensive, decentralized solution designed to modernize the way supply chains are managed. In traditional supply chain systems, data is often hidden within the private databases of different companies. This leads to a ‘trust gap’ where a Retailer might not know for sure if a product was manufactured using quality materials or if it was delayed at a warehouse.

Our system solves this by creating a ‘Digital Mirror’ of the physical supply chain on the Ethereum Blockchain. Every time a physical action happens—like a manufacturer finishing a batch of goods—a digital transaction is recorded.

Core Philosophies of the System:

**Transparency:** Every authorized participant can see the journey of an order. No one can hide a mistake or a delay.

**Immutability:** Once a status is updated (e.g., ‘Shipped’), it is written into the blockchain. It cannot be edited, deleted, or falsified by anyone, including the Admin.

**Efficiency through Automation:** By using a ‘Watcher Service,’ the system automatically updates local inventory counts whenever a blockchain event occurs, reducing manual data entry and human error.

**Security:** Access is controlled by cryptographic keys. Instead of just a password, users ‘sign’ their actions using a digital wallet (MetaMask).

## 3.2 System Architecture

The architecture of this system is ‘Hybrid.’ This means it combines the best of two worlds: the extreme security of Blockchain and the high speed of traditional Web Applications. The architecture is divided into **three** distinct layers:

### 1. The Presentation Layer (Frontend)

This is the ‘Face’ of the system. Built using React.js, it provides a modern, fast, and responsive interface.

**Role-Based Access:** The website detects who is logged in and shows only what they need to see. For example, a Supplier sees ‘Material Requests,’ while a Warehouse Officer sees ‘Incoming Shipments.’

**Blockchain Integration:** The frontend uses Ethers.js to talk to MetaMask. When a user clicks ‘Verify,’ the frontend triggers a popup in MetaMask for the user to authorize the transaction.

### 2. The Application & Synchronization Layer (Backend)

This layer acts as the ‘Middleman’ or the ‘Bridge.’

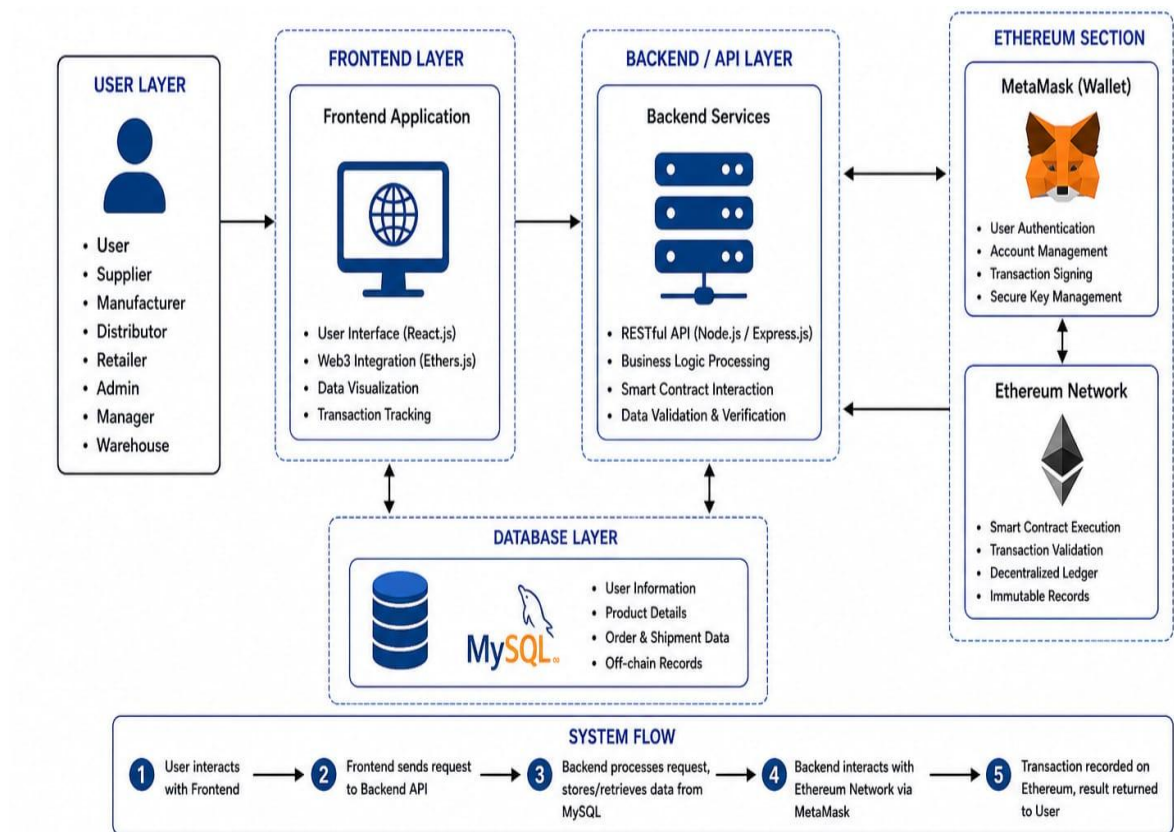
**Express API:** A Node.js server that handles tasks that don't need to be on the blockchain, such as user registration (names, phone numbers) and product details.

**The Watcher Service:** This is a specialized component that stays active 24/7. It listens to the Ethereum network for specific signals (Events). When the smart contract says ‘Status Updated,’ the Watcher catches that signal and instantly updates the MySQL database. This ensures the website always shows the latest stock levels without being slow.

### 3. The Data Layer (Blockchain & Database)

The Blockchain (On-Chain): Stores the ‘critical’ data—Order ID, Product Name, Quantities, Timestamps, and the Wallet Addresses of everyone who touched the product. This is the Permanent Ledger.

MySQL Database (Off-Chain): Stores ‘metadata’—user profiles, Bill of Materials (BoM) formulas, and a cached copy of the inventory. This makes searching and generating reports much faster and cheaper.



### 3.3 Module Description

The system is divided into four main modules, each handling a specific part of the business logic:

#### A. User & Security Module:

This module manages the seven roles of the system. It ensures that a Retailer cannot accidentally perform the job of a Manufacturer. It uses a Mapping Logic where a unique database ID is linked to a unique Blockchain Wallet Address. This creates a ‘Digital Identity’ for every stakeholder.

#### B. Smart Contract Module (Supply Chain .sol)

This is the ‘Brain’ of the system. It is a set of programmed rules that live on the blockchain.

State Management: It keeps track of 15 different statuses (from 0 to 14).

History Tracking: Every time a status changes, it pushes a new entry into a 'History Array.' This creates the timeline view seen on the dashboard.

Validation: It contains 'Modifiers' (rules) that check if an order exists before allowing anyone to update it.

### **C. Bill of Materials (BoM) & Product Module**

Managed primarily by the Admin(Owner), this module defines what a product is made of. For example, if the product is 'Biscuits,' the BoM might list '5g Sugar, 10g Flour.' When a Manager sees a demand for 100 packets of biscuits, this module automatically calculates that 500g of sugar and 1000g of flour are needed and sends a request to the Supplier.

### **D. Inventory & Logistics Module**

This module tracks the physical location of goods.

Warehouse Stock: Increases when the Manufacturer sends goods and the Warehouse verifies them.

Distributor Stock: Increases when goods are dispatched from the warehouse and received at the shop.

Retailer Confirmation: Decreases the distributor's 'available' stock once the final sale/delivery is verified.

## **3.4 Working Mechanism (The Step-by-Step Flow)**

To understand how the system functions in a real-world scenario, we follow the 'Life of an Order':

1. Demand Generation: A Retailer logs in, selects a product from the catalog, and enters a quantity. This creates a transaction on the blockchain with a unique Order ID.

2. Manager Review: The Manager sees this request. If the items aren't in the warehouse, the Manager clicks 'Instruct Supplier.' This moves the blockchain status to Raw Material Requested.
3. Sourcing: The Supplier sees the request for raw materials. Once they ship the flour/sugar, they click 'Ship Materials.'
4. Production: The Manufacturer receives the materials and clicks 'Start Manufacturing.' On the blockchain, the status changes to Manufacturing. Once the product is ready, they click 'Send to Warehouse.'
5. Logistics & Quality Check: The Warehouse Officer receives the physical goods. They click 'Verify & Audit.' This is a crucial step because the Watcher Service sees this and officially adds the items to the 'Warehouse Inventory' in the database.
6. Distribution: The Manager authorizes the move to the shop. The Distributor receives the goods, packages them, and ships them to the Retailer.
7. Final Closing: The Retailer receives the box, scans it, and clicks 'Final Receipt Verified.' The blockchain records the final timestamp, and the order is marked as 'Complete' in the history.

### **3.5 Technology Stack**

The technologies chosen for this project are industry standards, ensuring the system is professional, scalable, and secure.

#### **1. Blockchain Technologies**

**Solidity:** The programming language used to write the Smart Contract rules.

**Hardhat:** A development environment used to compile the code, run tests, and deploy the contract to the internet.

Sepolia Testnet: A public blockchain network that works exactly like the real Ethereum network but uses 'test coins' instead of real money.

MetaMask: A browser extension that acts as the user's digital identity and vault for authorized signatures.

## **2. Frontend Technologies**

React.js: A powerful framework for building fast user interfaces. It allows us to create a 'Single Page Application' where the page doesn't need to reload when clicking between tabs.

Tailwind CSS: A styling tool used to create the professional 'Dark Mode' and 'Clean' look of the dashboards.

Lucide React: A library of beautiful icons used to represent trucks, warehouses, and users visually.

## **3. Backend Technologies**

Node.js & Express: The engine that runs our API server. It is known for being extremely fast at handling multiple users at once.

Ethers.js: The 'Magic Cable' that connects our JavaScript code to the Ethereum Blockchain.

Sequelize: A tool (ORM) that lets us talk to our MySQL database using simple JavaScript code instead of complex SQL queries.

## **4. Database**

MySQL: A reliable and widely-used database that stores all our 'off-chain' information safely.



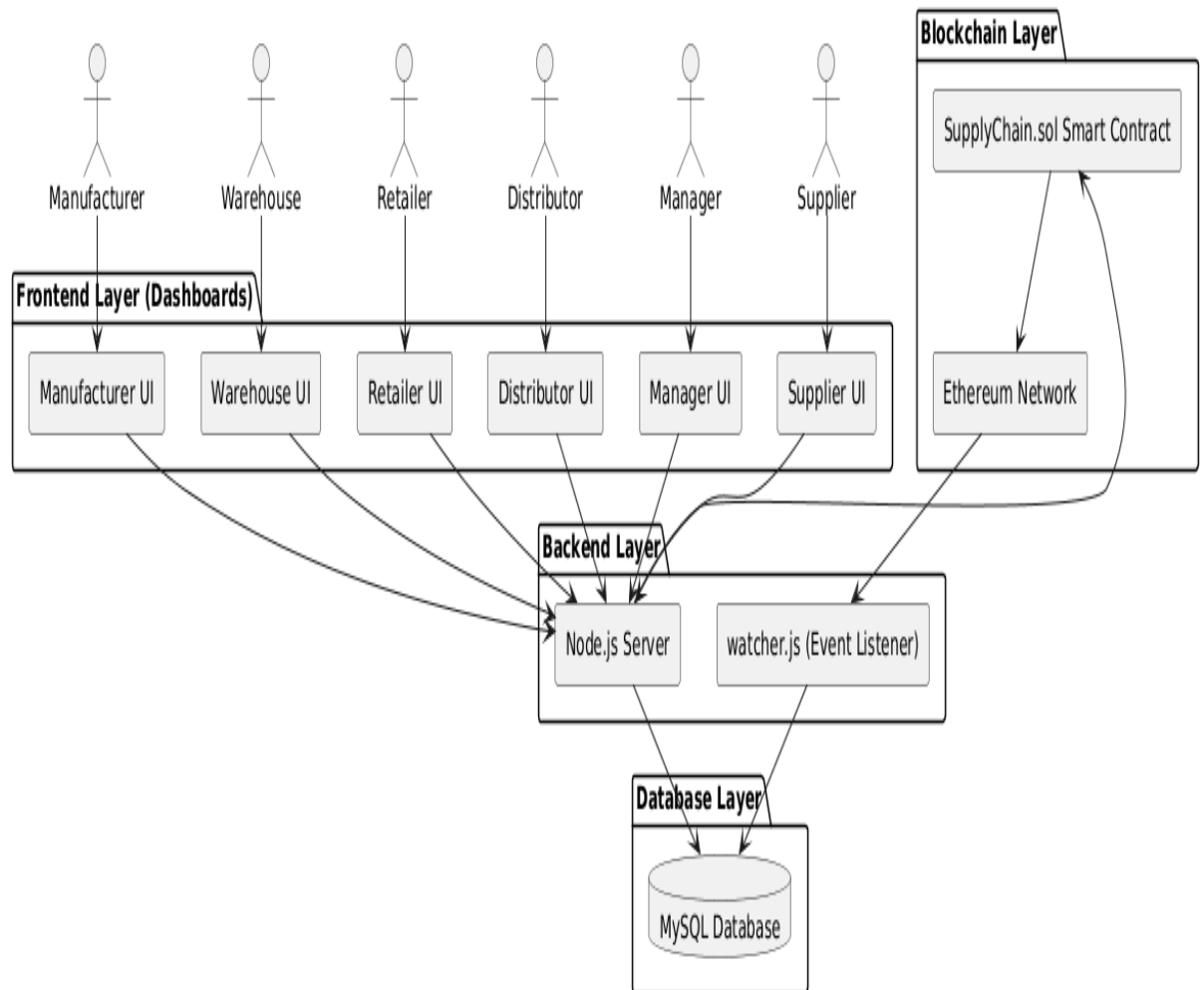
### 3.6 Diagrammatic Representation (ERD, Flowcharts, and DFD)

This chapter outlines the structural framework of the Blockchain-based Supply Chain Management system. It details how decentralized ledger technology integrates with traditional database systems to ensure transparency, traceability, and data integrity across the supply chain.

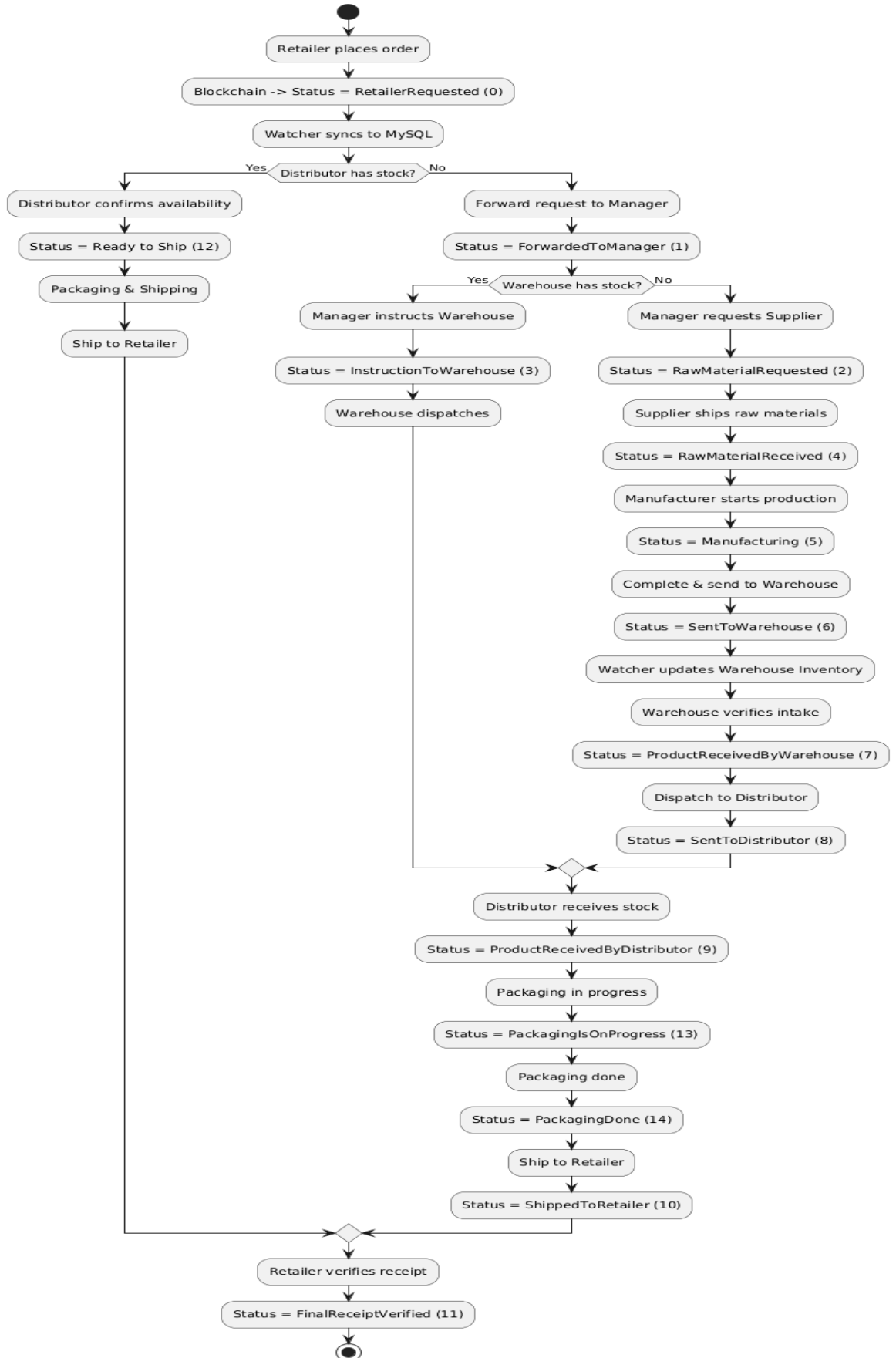
#### Flow Chart Diagram Of System:-



## System Architecture:-



## UML Diagram:-



## **CHAPTER 4**

### **Integrating Blockchain in Supply Chain Management**

#### **4.1 Benefits of Integrating Blockchain in Your Specific SCM**

The traditional supply chain is often called a ‘Black Box’ because once a product leaves a factory, it becomes very difficult for other partners to know exactly what is happening to it. By integrating the Ethereum Blockchain into our specific 7-role system, we have turned this ‘Black Box’ into a ‘Glass Box’—where everything is visible and verified.

##### **A. Establishing an Absolute ‘Source of Truth’**

In our project, we have many stakeholders: Suppliers, Manufacturers, Managers, Warehouse Officers, Distributors, and Retailers. In a normal system, each of these people would keep their own records in their own Excel sheets or private databases. If a Manufacturer claims they sent 100 units but the Warehouse says they only received 90, it leads to a long dispute.

In our system, every action is recorded on a shared ledger. Since everyone is looking at the same Smart Contract, there is no room for disagreement. If the blockchain says ‘Status: Sent to Warehouse,’ it is a fact that everyone in the network accepts instantly.

##### **B. Cryptographic Accountability**

Standard systems use ‘Usernames and Passwords.’ These can be shared, stolen, or used by anyone. Our system uses Public Key Infrastructure (PKI) via MetaMask. Every time a Warehouse Officer clicks ‘Verify Receipt,’ they are not just clicking a button; they are digitally signing a transaction with a unique private key.

The blockchain records the specific Wallet Address of the person who performed the action. This means if a batch of materials is marked as ‘Received’ but is actually missing, we have a permanent, un-erasable record of exactly which wallet address authorized that entry. This creates a level of individual accountability that is impossible in traditional databases.

### **C. Automation of the Bill of Materials (BoM)**

One of the most unique benefits of our specific implementation is the BoM Module. Normally, a Manufacturer has to manually calculate how much raw material is needed for an order. This manual process is slow and prone to errors. Our system automates this. When a Retailer requests a product, the system looks up the 'Formula' (the BoM) stored in our database and calculates the requirements. This information is then passed through the supply chain. This automation ensures that the Supplier knows exactly what to ship and the Manufacturer knows exactly what to use, reducing waste and saving time.

### **D. Data Immutability (The 'Un-Changeable' Rule)**

The word 'Immutable' simply means 'cannot be changed.' In a traditional database (like MySQL alone), a clever hacker or a dishonest Admin could go into the database and change the 'Date of Delivery' to hide a delay.

In our project, once a status update is sent to the Sepolia Testnet, it is carved in stone. Not even the System Admin can go back and change a timestamp or delete a transaction. This ensures that the history of the product is 100% honest and accurate.

## **4.2 Challenges Encountered During Integration**

Building a system that talks to both a high-speed database and a decentralized blockchain is technically difficult. We encountered three major 'speed bumps' during development:

### **4.2.1 The Constraint of Gas Fees**

On the Ethereum network, nothing is free. Every time we write data (like creating an order or updating a status), we must pay a fee to the network miners called 'Gas.'

The Problem: Storing large amounts of data on the blockchain is extremely expensive. If we tried to store a user's full home address, phone number, and profile picture on the blockchain, the cost per order would be too high.

Our Solution: We designed a 'Lightweight' Smart Contract. We only store the 'Critical Data' on the blockchain (Order IDs, Status Codes, and Wallet Addresses). All the 'Heavy

Data' (Names, Phone Numbers, and BoM Details) is stored in our MySQL Database. This hybrid approach keeps the system affordable while remaining secure.

#### **4.2.2 Dealing with Network Latency**

'Latency' is just a fancy word for waiting time.

The Problem: When you post a photo on Instagram, it is instant. But when you send a transaction to the blockchain, you have to wait for the network to 'mine' the block. This usually takes 12 to 15 seconds on the Sepolia network.

Our Solution: We improved the 'User Experience' (UX) by adding visual feedback. When a user clicks 'Ship Materials,' the dashboard shows a 'Processing...' spinner. This prevents the user from clicking the button twice and ensures they understand that the global network is currently verifying their signature.

#### **4.2.3 The Synchronization Challenge (The Watcher)**

Since we have data in two places (Blockchain and MySQL), they must always stay in sync.

The Problem: If a Manufacturer updates a status on the blockchain, the Retailer's dashboard (which reads from our MySQL database for speed) needs to show that change immediately.

Our Solution: We developed a specialized 'Watcher Service' in Node.js. This service acts like a 24/7 security guard that listens to the blockchain. The moment the smart contract emits a 'Status Updated' signal, the Watcher catches it and instantly updates the MySQL inventory tables. This was the hardest part of the project to build, as it requires a perfect connection between the backend and the Ethereum network.

### **4.3 Analyzing the 15-Status Order Lifecycle in the Ledger**

To provide total visibility, we did not just use 'Pending' and 'Completed' statuses. Instead, we broke the product journey into 15 distinct steps. This allows a Manager to see exactly where a bottleneck is occurring.

1. Status 0–1 (The Request): The Retailer asks for goods. If the Distributor is out of stock, the order is forwarded to the Manager.
2. Status 2–4 (The Sourcing): The Manager asks the Supplier for materials. The materials are shipped and then ‘Received’ by the Manufacturer. Each of these is a separate on-chain event.
3. Status 5–6 (The Factory): The Manufacturer starts ‘Manufacturing.’ When finished, the status changes to ‘Sent to Warehouse.’
4. Status 7–8 (The Hub): The Warehouse Officer verifies the boxes. The Manager then gives a specific ‘Instruction’ to dispatch them to the Distributor.
5. Status 9, 12–14 (The Shop): The Distributor receives the product. They mark it as ‘Available.’ Then, they track the internal packaging progress (‘In Progress’ -> ‘Done’).
6. Status 10–11 (The Final Mile): The product is ‘Shipped to Retailer.’ The lifecycle ends only when the Retailer verifies the ‘Final Receipt.’

Why 15 statuses? By having 15 steps, we can answer questions like: ‘Is the delay happening in the factory (Status 5) or in the warehouse (Status 7)?’ This level of detail is what makes this project a professional-grade SCM tool.

## 4.4 Comparison: Blockchain SCM vs. Traditional ERP Systems

To understand the value of this project, we must compare it to ERP (Enterprise Resource Planning) systems, which are currently used by most big companies.

| Feature      | Traditional ERP System                     | Our Blockchain System                   |
|--------------|--------------------------------------------|-----------------------------------------|
| Control      | Centralized (One company owns the data).   | Decentralized (Everyone owns the data). |
| Data Editing | An Admin can change or delete old records. | Impossible to change or delete records. |
| Trust Factor | Partners must trust the ERP owner          | Partners trust the Code and Math        |
| Cost         | Very high monthly subscription fees        | Low setup cost + small transaction fees |
| Transparency | Partners only see a 'summarized' report    | Partners see the Live Transaction Hash  |



## CHAPTER 5

### Project Implementation and Development

Introduction Project implementation is the phase where the theoretical design is converted into a functional software system. For this project, development was divided into three main areas: the Blockchain (Smart Contract), the Backend (Server and Watcher), and the Frontend (User Interface). This chapter details the tools, logic, and steps taken to build the Decentralized Supply Chain system.

#### 5.1 Designing the Smart Contract (SupplyChain.sol)

The ‘heart’ of our project is the SupplyChain.sol file. This is a Smart Contract written in Solidity.

The Order Struct: We designed a ‘Digital Passport’ for every product batch called a struct. It stores essential information like the Product Name, Quantity, Price, and the unique ID of the Retailer, Manufacturer, and Supplier.

The Status Engine: We implemented an enum with 15 specific statuses (Status 0 to 14). This allows the contract to track exactly where a product is at any second.

The History Array: Every time a status changes (e.g., from ‘Manufacturing’ to ‘Sent to Warehouse’), the contract creates a History Entry. This includes a timestamp and the wallet address of the person who made the change. This ensures that the history of the product is permanent and can never be deleted.

#### 5.2 Setting up the Local Development Environment

Before writing code, we had to prepare a ‘Digital Laboratory.’

Node.js & NPM: We installed Node.js to run our JavaScript code and manage all the libraries (like Ethers.js and Express).

Hardhat: This is the most important tool for blockchain development. It allows us to compile our Solidity code and check for errors. It also provides a ‘Local Blockchain’ so we can test the system on our computer before sending it to the internet.

Environment Variables (.env): We used a .env file to store sensitive data like our Private Keys and API URLs. This keeps our blockchain account safe from hackers.

MySQL & Sequelize: We set up a local MySQL server to store user profiles and inventory counts. We used Sequelize as a helper to let our JavaScript code talk to the database easily.

### 5.3 Defining Network Participants (The 7-Role Logic)

A unique feature of this project is its complex role-based system. We defined seven distinct participants:

1. **Admin:** Manages the entire network and registers new users.
2. **Supplier:** Provides raw materials (like flour or sugar).
3. **Manufacturer:** Converts raw materials into finished products.
4. **Manager:** The ‘Director’ who gives instructions to the warehouse and supplier.
5. **Warehouse Officer:** Audits the stock and verifies incoming goods.
6. **Distributor:** Handles the packaging and sends products to shops.
7. **Retailer:** The shopkeeper who places orders and confirms final delivery.

We used a Mapping Logic to link each user’s database profile to their unique Ethereum Wallet Address. This ensures that if you are logged in as a ‘Retailer,’ the system will only let you perform ‘Retailer’ actions.

## **5.4 Backend Development: The ‘Watcher’ Service Logic**

The ‘Watcher’ (watcher.js) is the most technically advanced part of our backend

Normally, a website has to ‘ask’ the blockchain for updates, which is very slow. To solve this, we built a service that listens instead of asking.

Event Listening: When the Smart Contract emits a Status Updated signal on the blockchain, our Watcher service catches it instantly.

Automatic Inventory Update: If the Watcher sees a status change to ‘Sent to Warehouse,’ it automatically runs code to increase the quantity of that product in our MySQL inventory table. This means the website always shows the correct stock levels without the user having to refresh the page.

## **5.5 Database Integration (MySQL and Sequelize Models)**

We used a Hybrid Data Model. Not everything is stored on the blockchain because blockchain storage is expensive.

Off-Chain Data: We use MySQL to store ‘heavy’ data like User Names, Passwords, Phone Numbers, and the Bill of Materials (BoM).

Inventory Tables: We created two specific tables: inventory (for the central warehouse) and distributor inventory (for the shop).

Sequelize Models: In models.js, we defined these tables as JavaScript objects. This allowed us to perform tasks like `Inventory.update()` or `User.create()` with very simple code, making the development faster and less prone to SQL errors.

## 5.6 Frontend Development: Role-Based Dashboards

The user interface was built using React and Tailwind CSS. We focused on making it look like a professional business tool.

**The AppContext:** We created a central ‘Brain’ for the frontend (AppContext.jsx). It handles the connection to MetaMask and shares the latest order data with all the dashboards.

**Dynamic UI:** Depending on who logs in, the website changes.

The Manufacturer sees a ‘Start Production’ button.

The Distributor sees a ‘View Bill’ and ‘Ship’ button.

The Admin sees a ‘Register User’ form.

**Live Pipeline:** We built a visual timeline for every order. It uses icons (from Lucide React) and colors to show exactly which stages are completed and which are pending.

## 5.7 Testing and Debugging on Sepolia Testnet

Once the local version was working, we moved to the Sepolia Testnet. This is a public blockchain that behaves exactly like the real Ethereum network.

**Faucet Coins:** We used a ‘Faucet’ to get free test ETH to pay for our gas fees.

**Debugging:** We used console.log and the Hardhat terminal to track transactions. We carefully checked that when a status changed on the Sepolia network, our Watcher Service correctly updated our local database.

**Error Handling:** We added rules to our Smart Contract to prevent mistakes. For example, the contract will ‘Revert’ (cancel) a transaction if a Retailer tries to mark an order as ‘Manufactured.’

## 5.8 Deployment Strategy and Contract Interaction

Deploying the contract means ‘uploading’ it to the blockchain so anyone in the world can use it

1. **Deployment Script:** We used `deploy.js` to send our contract to Sepolia.
2. **Contract Address:** Once deployed, the blockchain gave us a unique Contract Address. We saved this address in our `.env` files for both the frontend and backend.
3. **The ABI:** We used the Application Binary Interface (ABI), which is like a ‘User Manual’ that tells our JavaScript code how to talk to the Smart Contract functions like `createOrder` and `updateStatus`.

## 5.9 Monitoring Data Consistency via On-chain Events

To ensure the system never crashes or shows wrong data, we implemented Consistency Monitoring.

**Dual-Verification:** Every 3 seconds, the React frontend ‘polls’ (checks) the MySQL database for new inventory numbers. At the same time, it remains connected to the blockchain for live updates.

**Integrity Checks:** If the Watcher service misses an event due to an internet failure, we built a ‘Manual Sync’ button. This button scans the entire blockchain history and re-updates the database to make sure the numbers are 100% correct.

# CHAPTER 6

## Specific Features of the Developed System

### 6.1 Introduction

The decentralized Supply Chain Management system developed in this project is not just a simple tracking tool; it is a sophisticated ecosystem of integrated modules. While the blockchain provides the foundation of security, the specific features built on top of it—such as the Live Pipeline, Automated BoM, and the Watcher Service—are what make the system practical for real-world industrial use. This chapter explores these four core features in detail.

### 6.2 Different Dashboard For All Stakeholder

To ensure operational efficiency, the system provides five customized dashboards, each designed for the specific needs of the stakeholder:

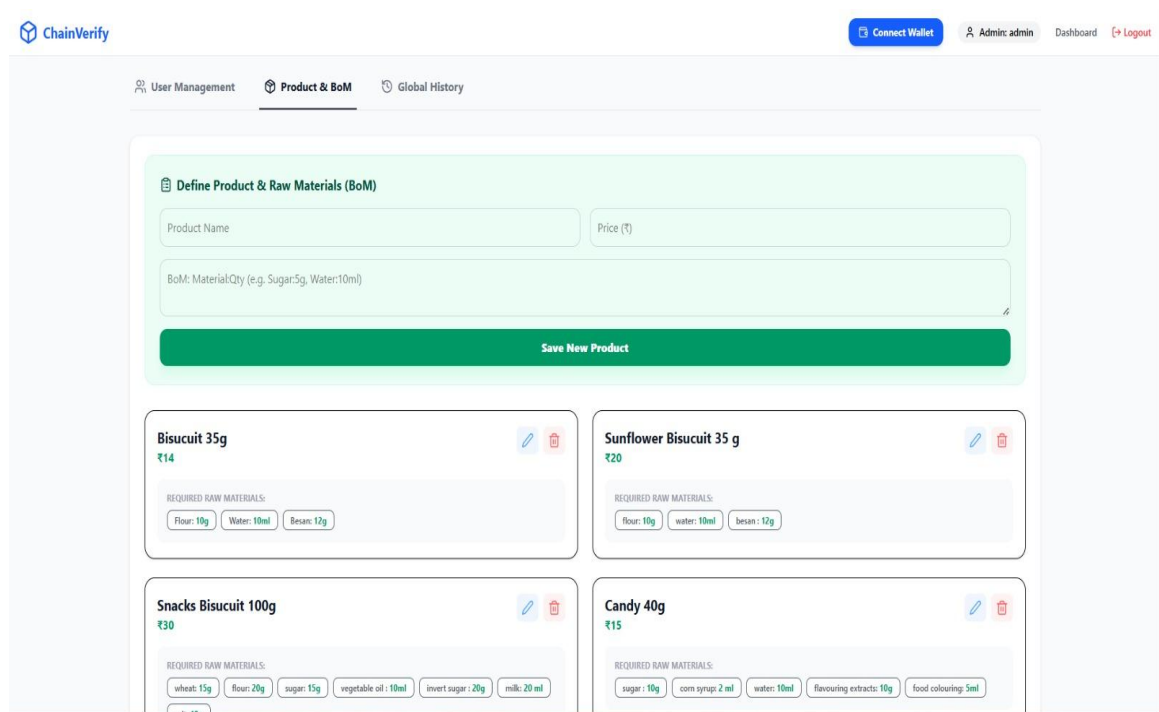


Figure 6.2.1: Admin Dashboard; Details of Product and its raw material

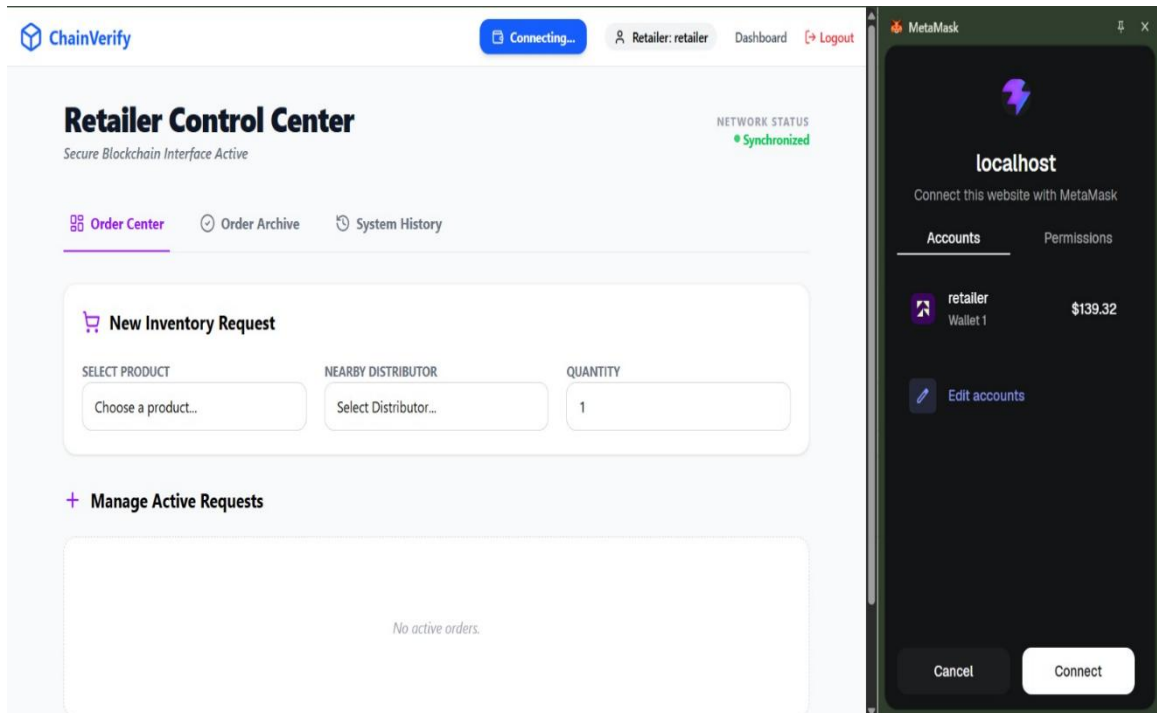


Figure 6.2.2: Retailer Dashboard.

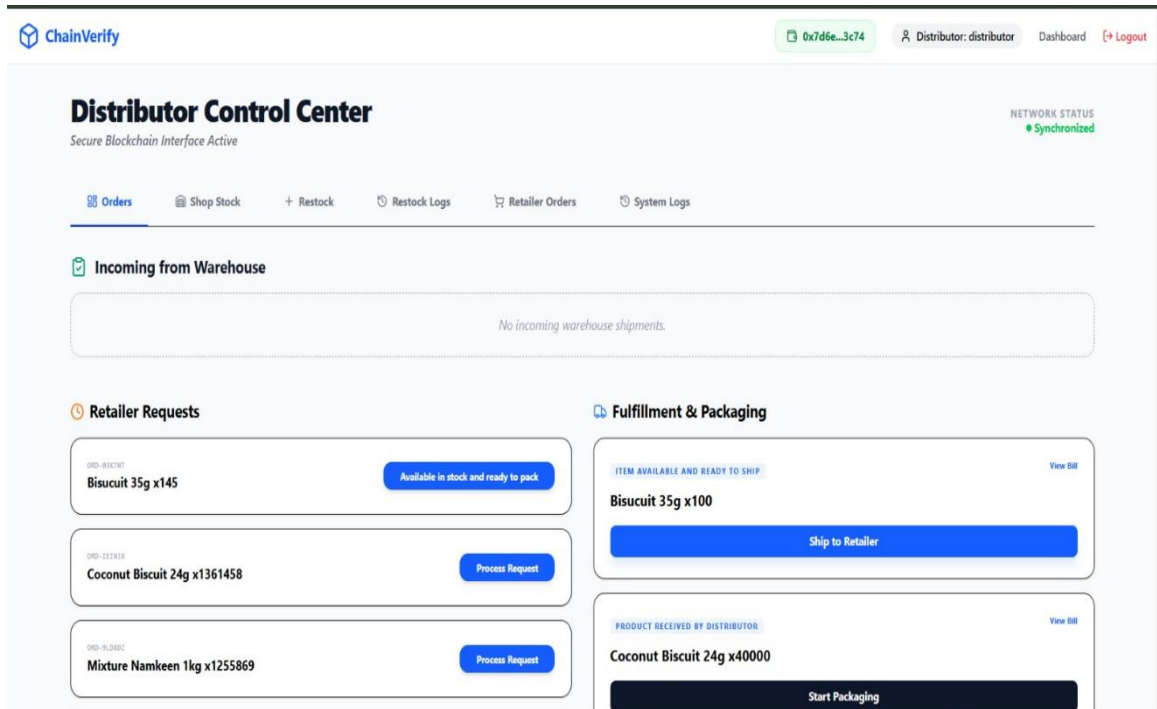


Figure 6.2.3: Distributor Dashboard.

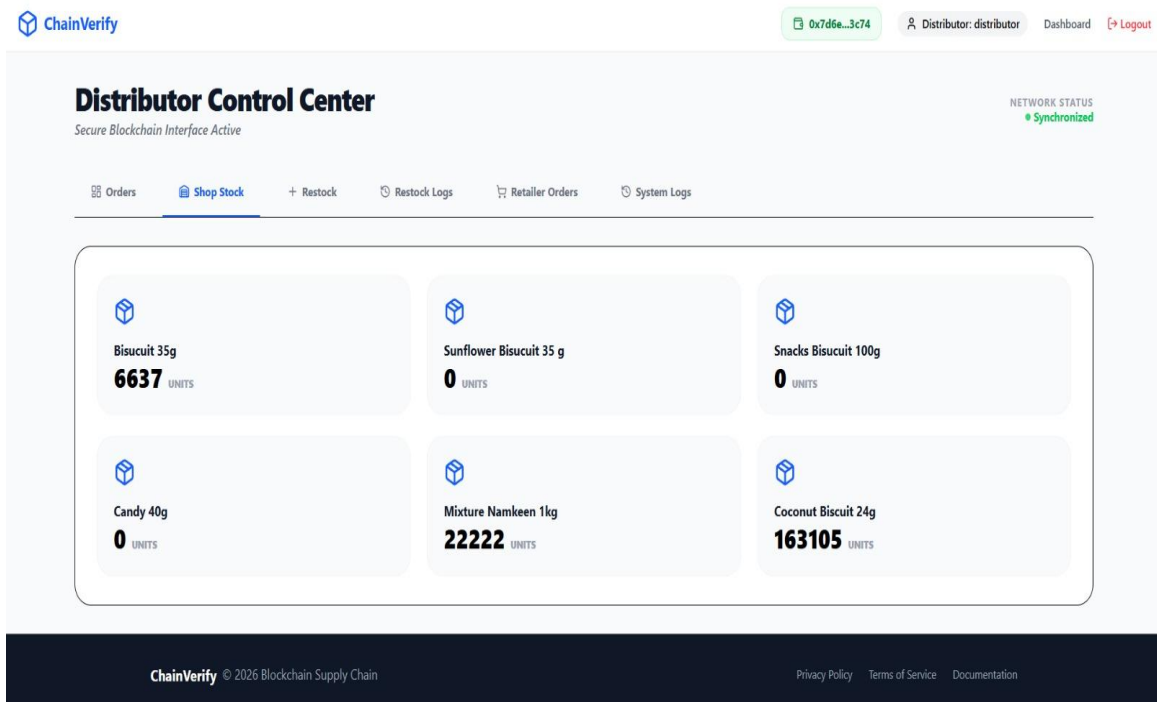


Figure 6.2.4: Retailer Dashboard: Showing Available Stock.

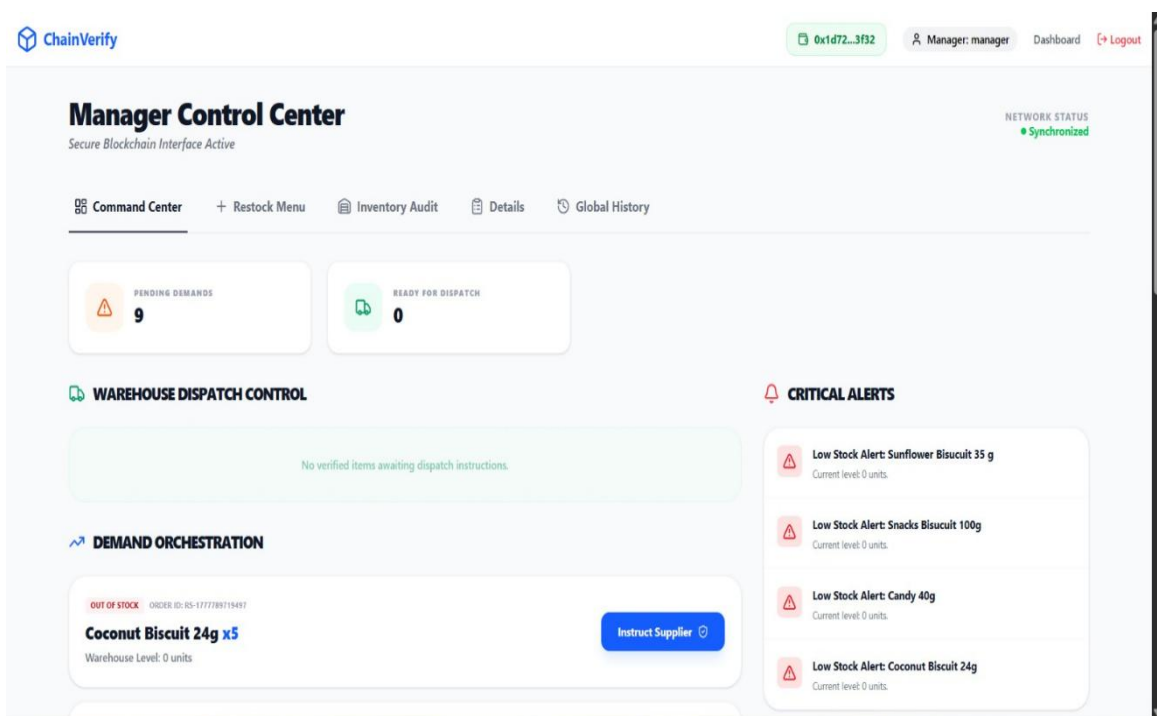


Figure 6.2.5: Manager Dashboard:



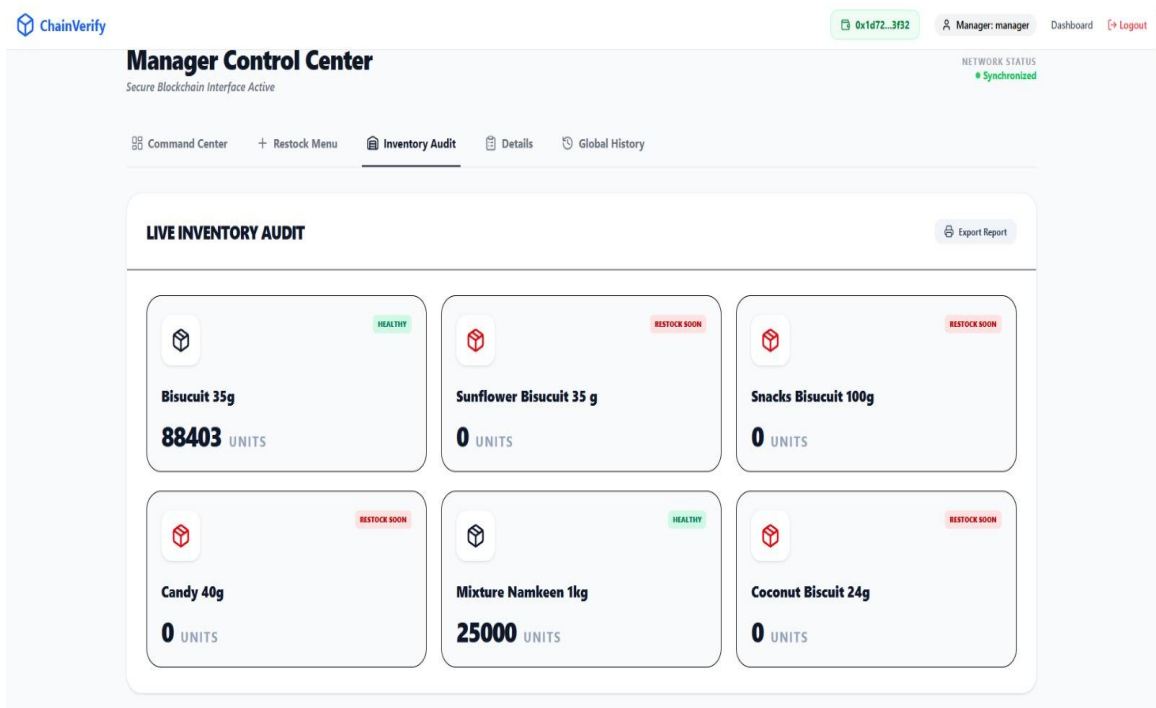


Figure 6.2.6: Manager Dashboard: Showing available stock in warehouse

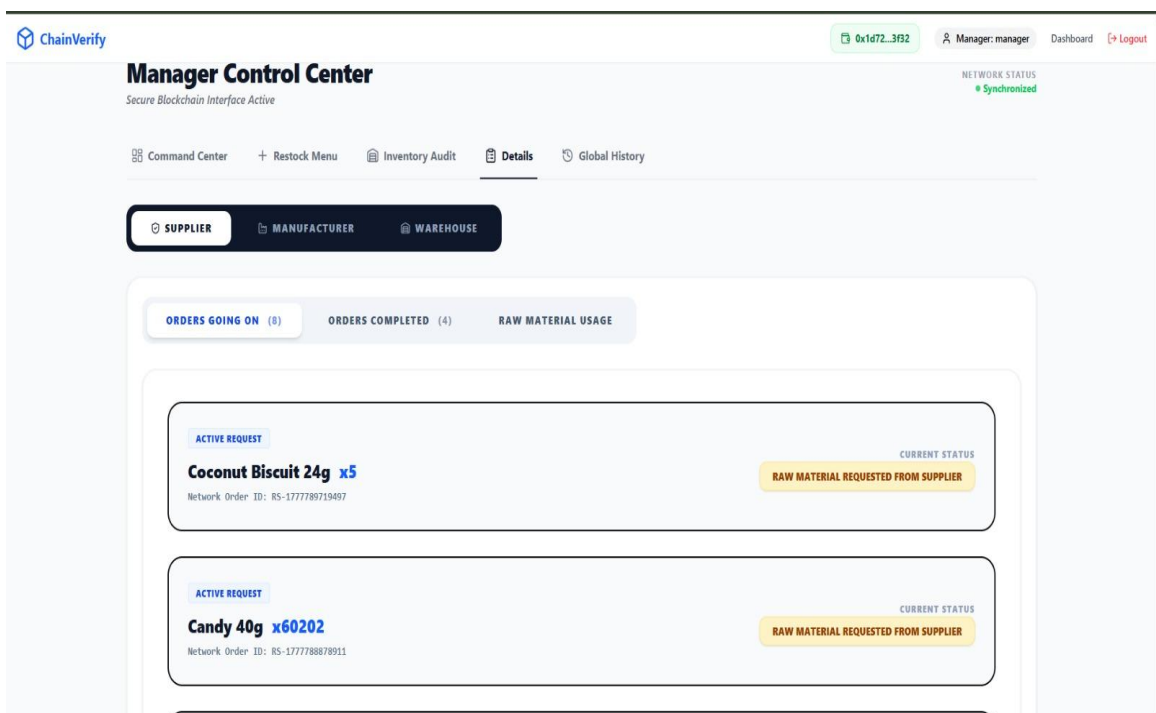


Figure 6.2.7: Manager Dashboard: Monitoring the Supplier

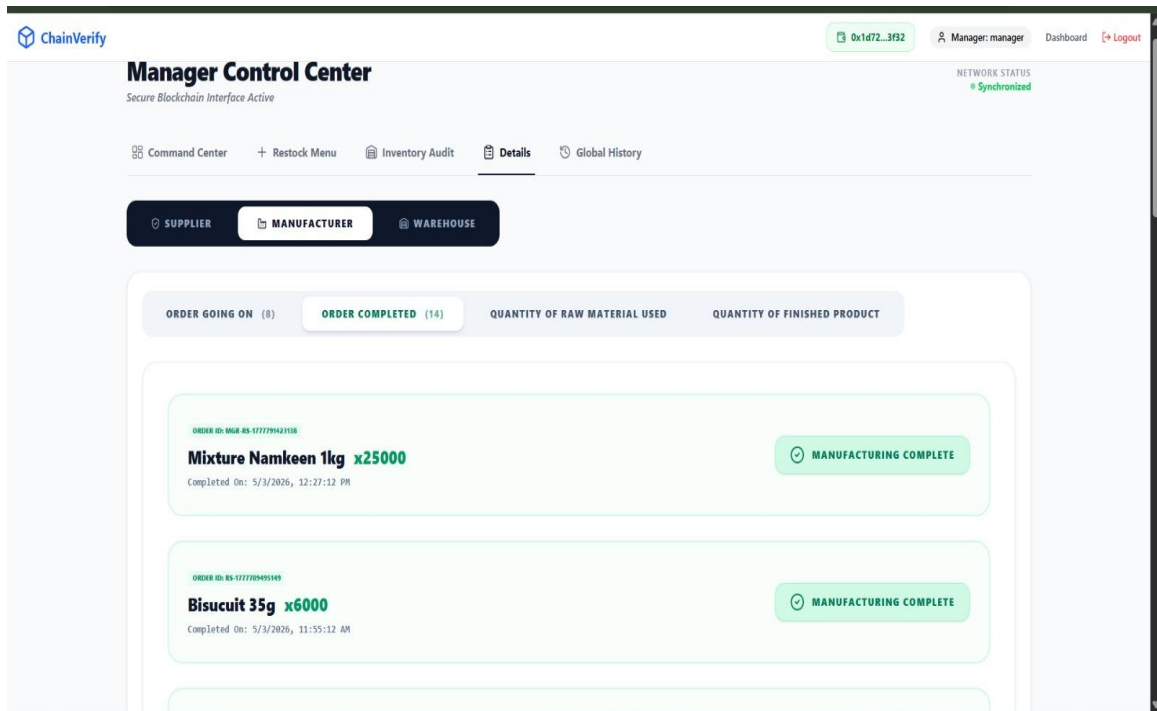


Figure 6.2.8: Manager Dashboard: Monitoring the Manufacturer

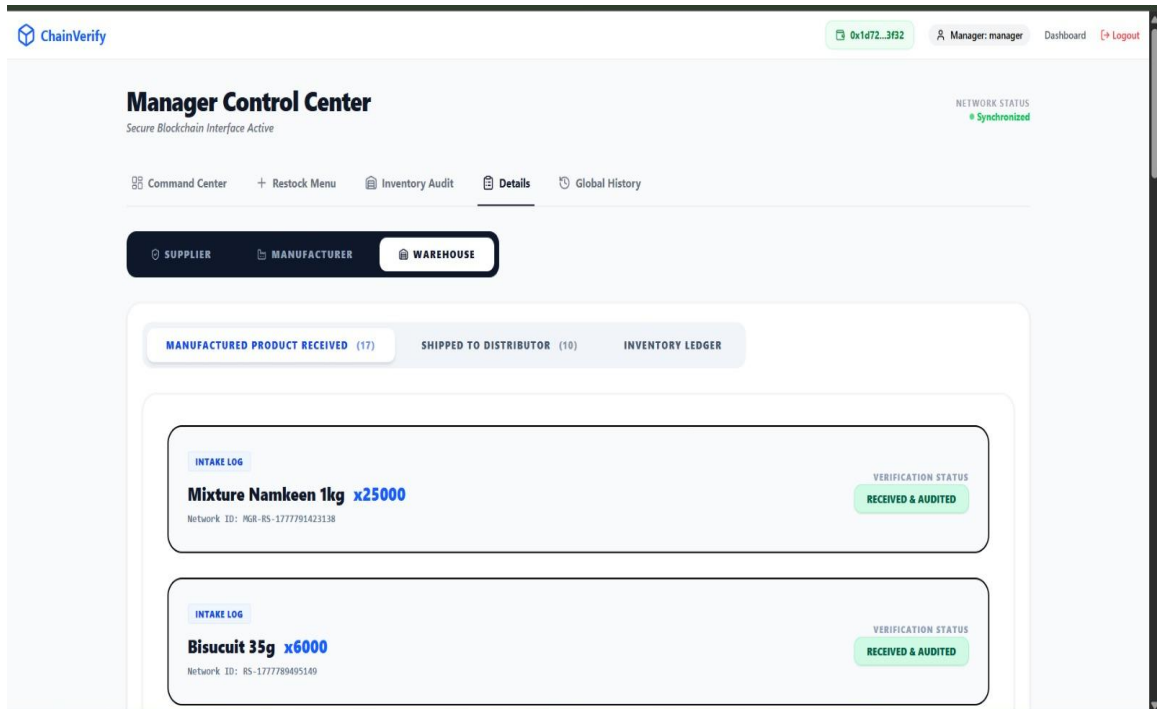


Figure 6.2.9: Manager Dashboard: Monitoring the Warehouse

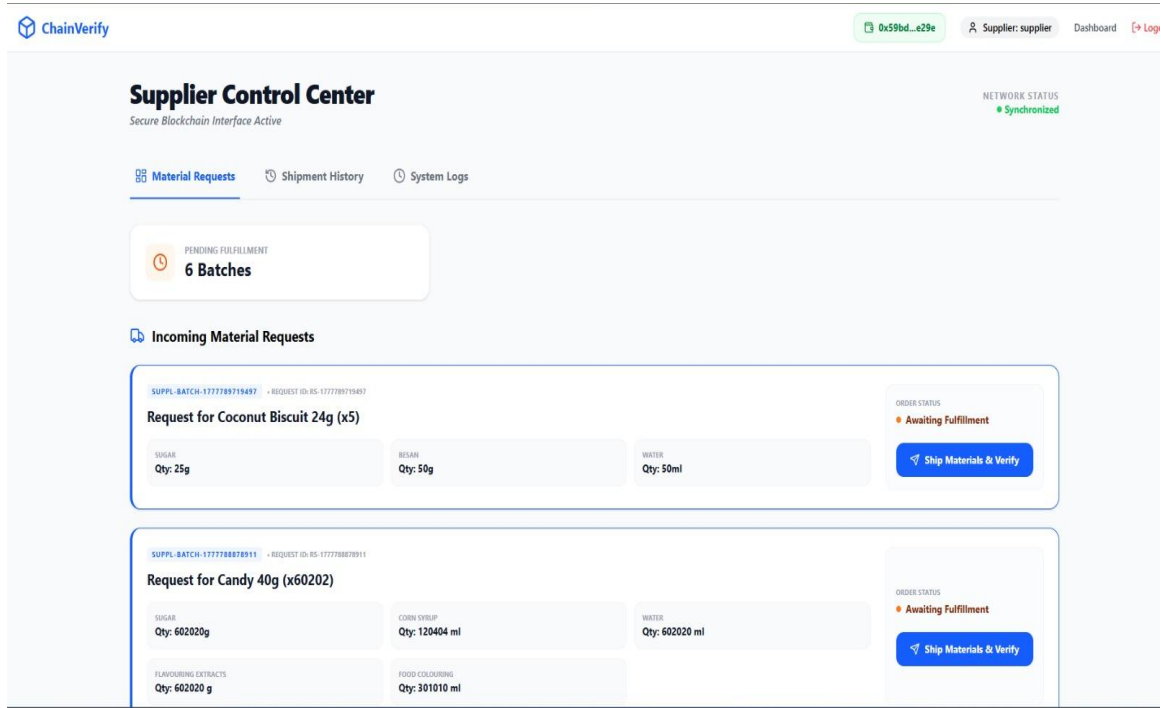


Figure 6.2.10: Supplier Dashboard

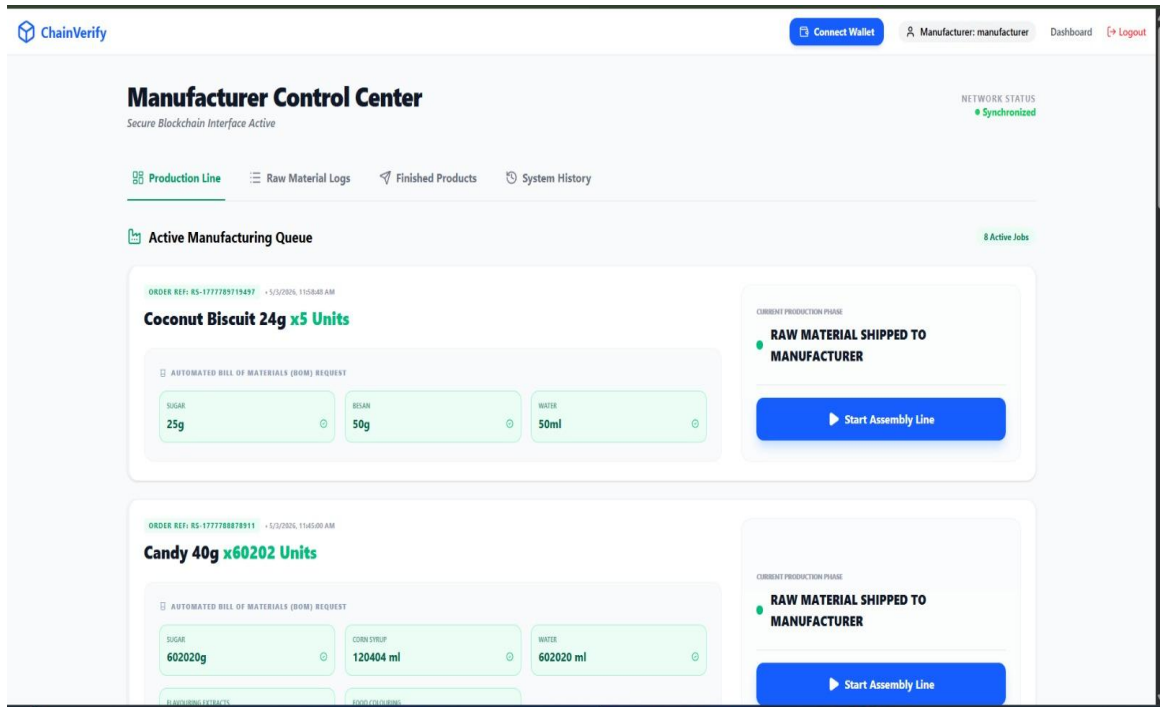


Figure 6.2.11: Manufacturer Dashboard

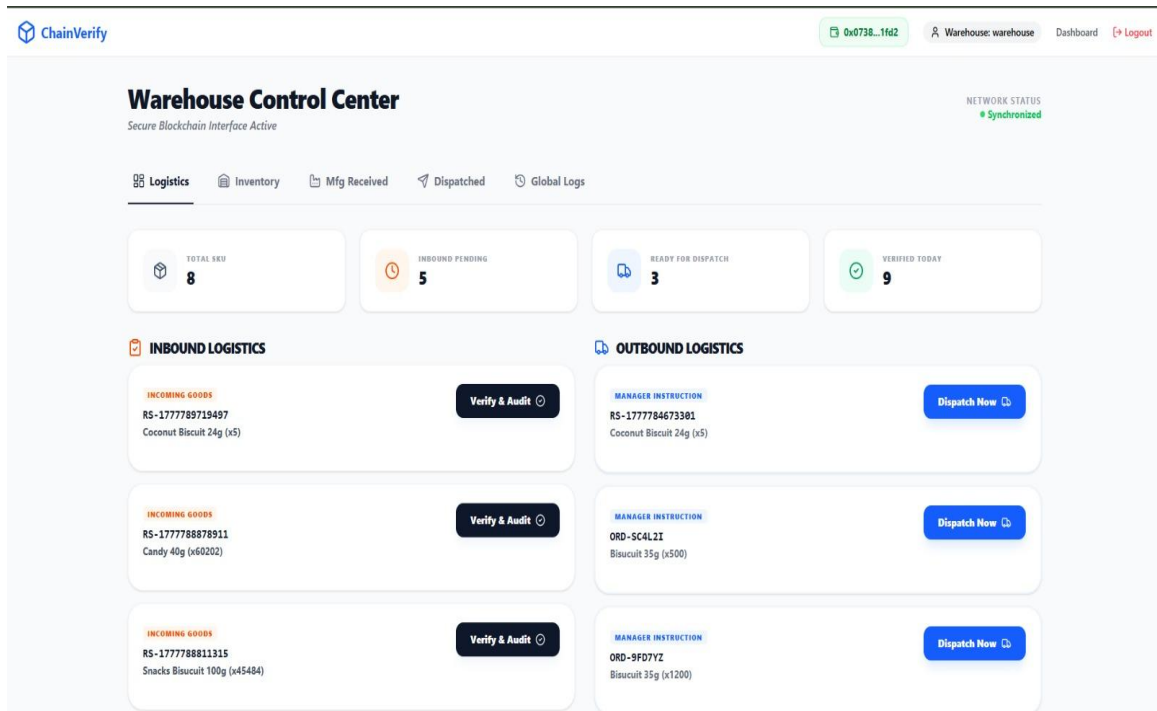


Figure 6.2.12: Warehouse Dashboard

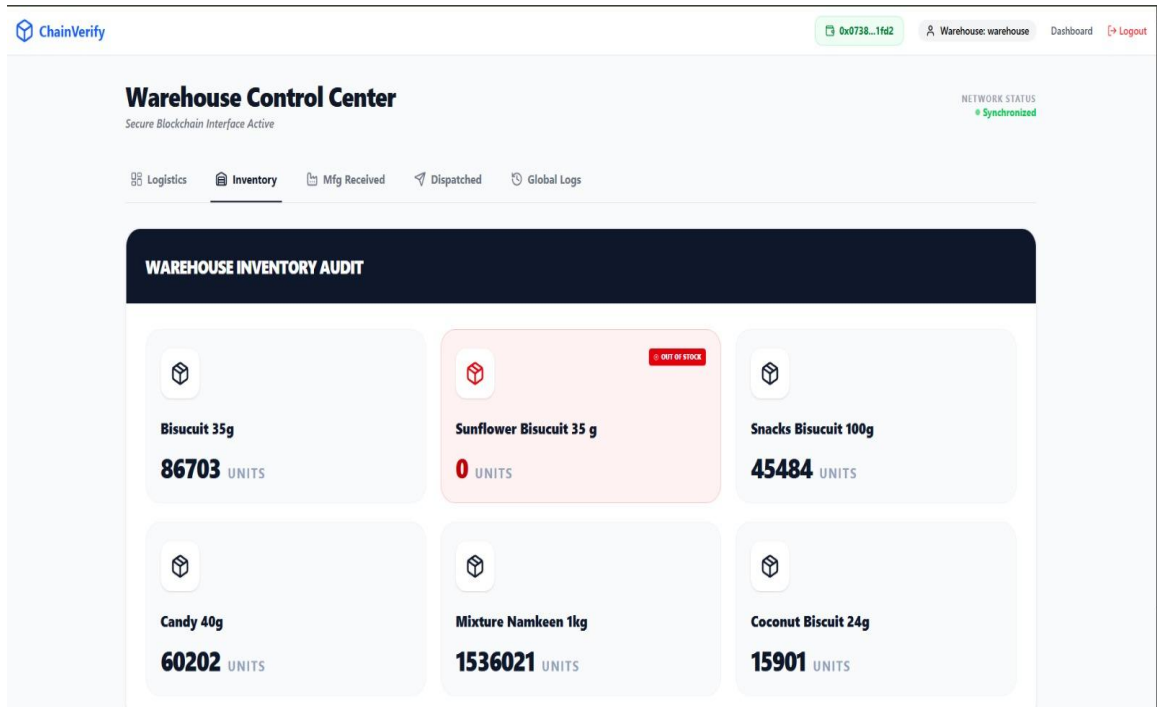


Figure 6.2.13: Warehouse Dashboard: Showing Stock

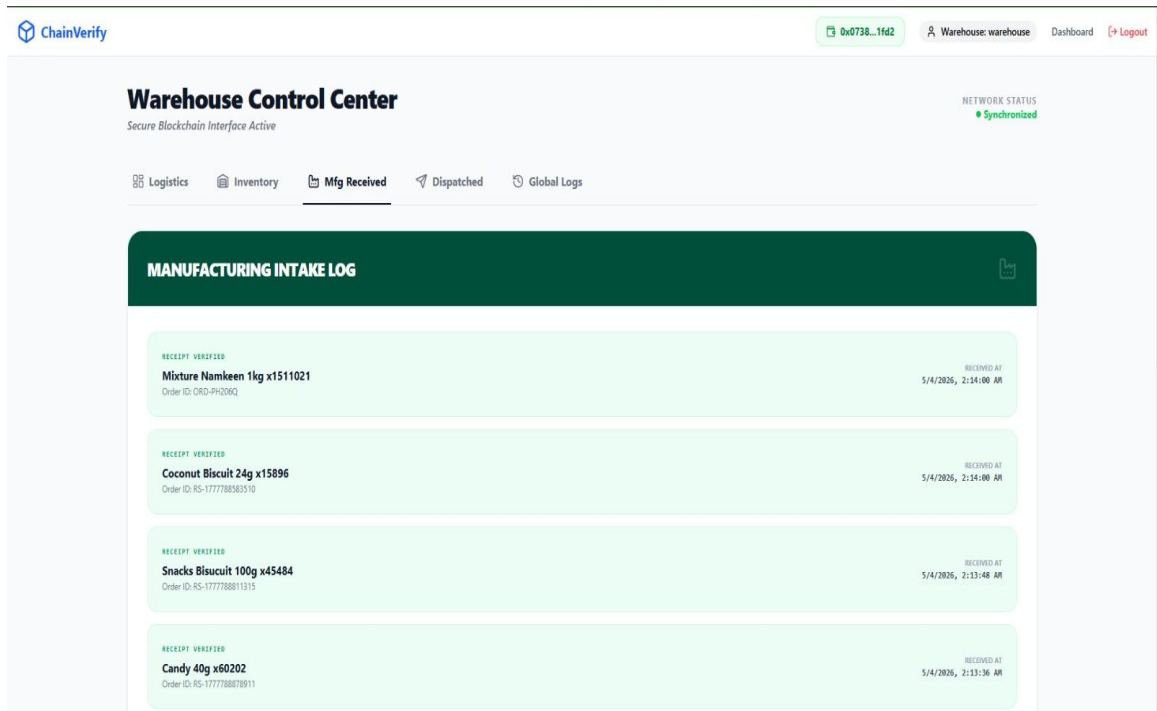


Figure 6.2.14: Warehouse Dashboard: Showing Mfg Received

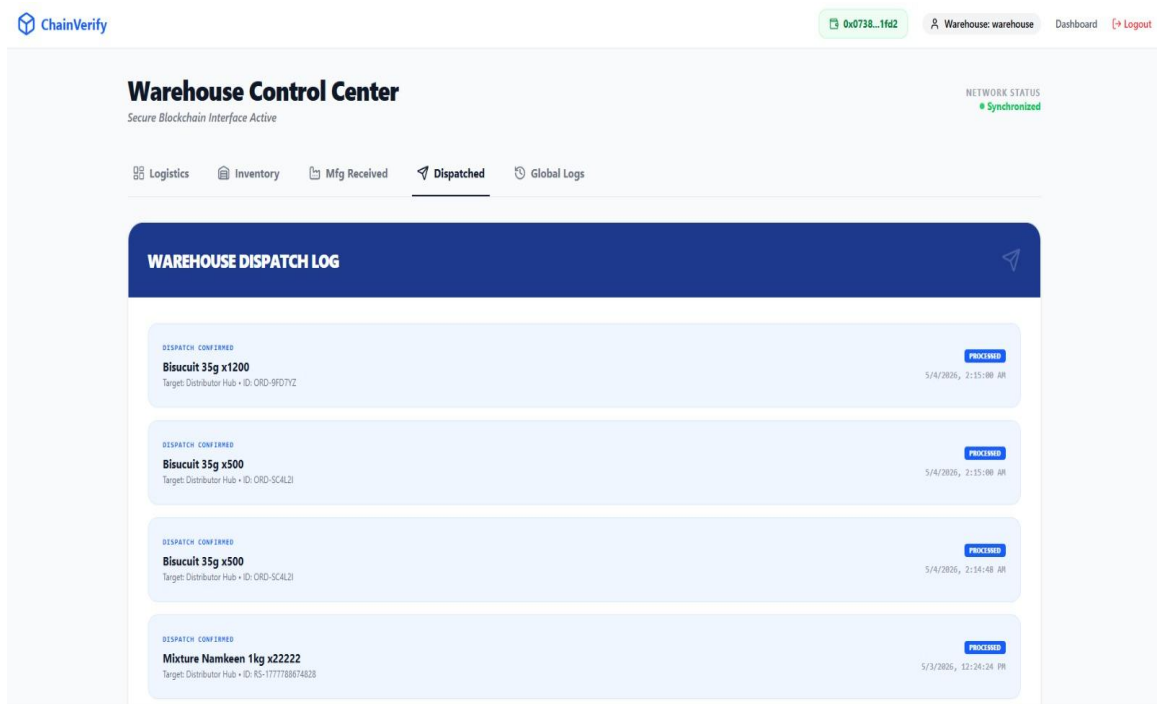


Figure 6.2.15: Warehouse Dashboard: Showing Dispatched item

## 6.3 Real-time Tracking and Traceability (Live Pipeline View)

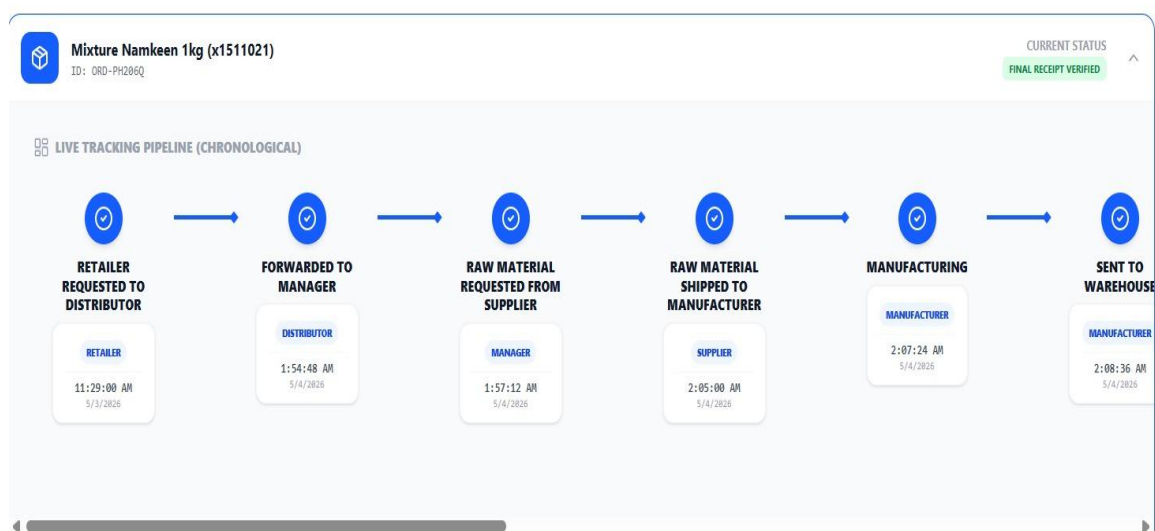
In traditional logistics, tracking a product often involves looking at a ‘text-only’ log that can be confusing or outdated. Our system introduces a Live Tracking Pipeline, which is a visual representation of the product's entire journey from the first request to final delivery.

### 6.3.1 How the Pipeline Works

The pipeline is a dynamic component built in React. It pulls data directly from the order History mapping in the SupplyChain.sol smart contract.

The Chain of Events: Every time a role (like the Manufacturer) updates a status, the blockchain records the event. The frontend identifies which of the 15 statuses are ‘Completed’ and which are ‘Next.’

Visual Logic: Completed steps are highlighted in solid colors (e.g., Green or Blue), and the current active step is shown with an ‘Animated Pulse.’ This allows any user to instantly see if an order is stuck in ‘Manufacturing’ or ‘Packaging’ without reading complex data.



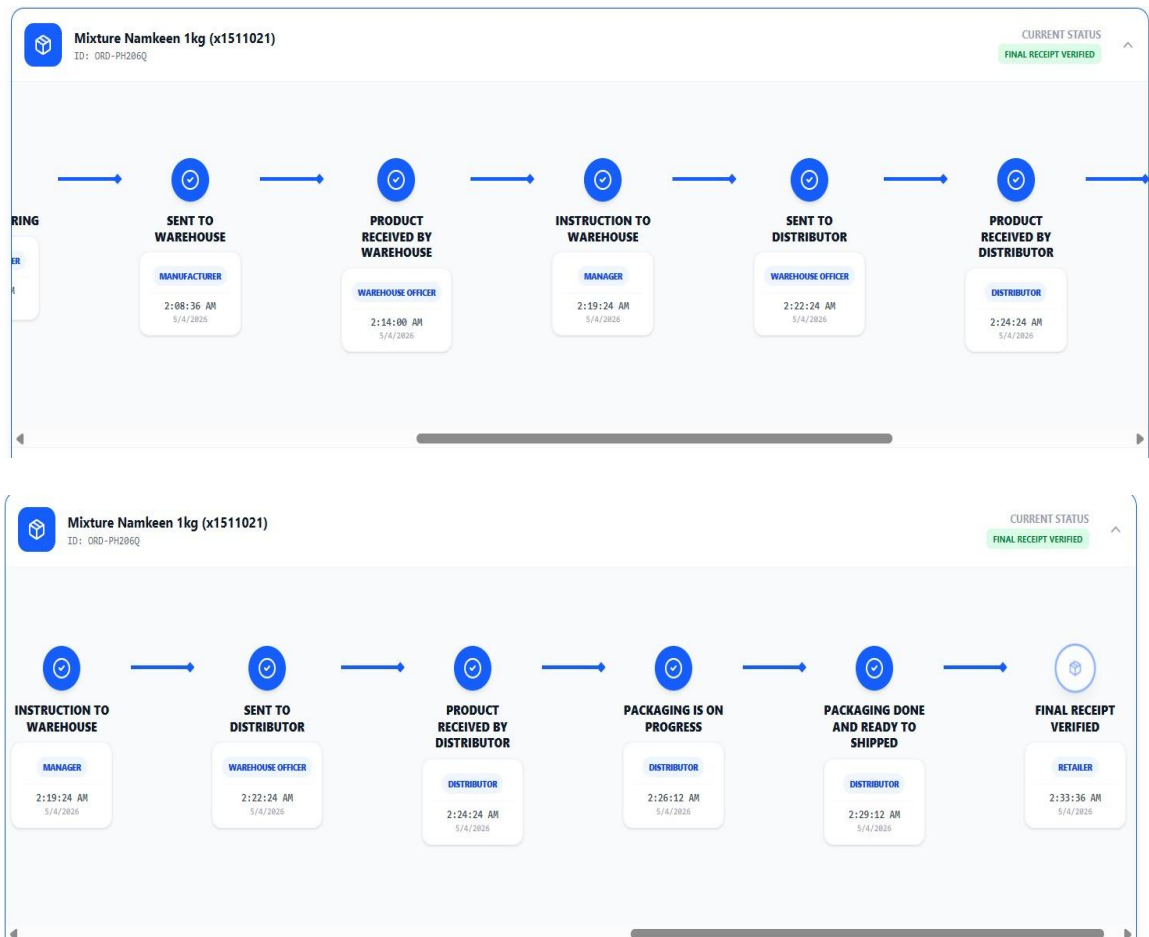


Figure: 6.3.1: Live Pipelining

### 6.3.2 The ‘Immutable Audit Trail’

Beneath the visual pipeline is the History View. This is a detailed list of every ‘handshake’ that has occurred.

**Actor Verification:** Each entry shows the exact Ethereum Wallet Address of the person who moved the product.

**Role Identification:** It displays the role (e.g., ‘Warehouse Officer’) to ensure accountability.

**Timestamping:** The time is fetched from the blockchain’s block-time, making it impossible for anyone to lie about when a product was shipped or received.

## **6.4 Automated Bill of Materials (BoM) and Raw Material Requests**

One of the most powerful features of our system is how it handles production requirements. Instead of the Manufacturer guessing what materials they need, the system automates the calculation.

### **6.4.1 Defining the Product Formula**

In the Admin Dashboard, the Admin defines a 'Bill of Materials' (BoM) for every product. For example:

Product: Packet of Biscuits.

BoM: 5g Sugar, 10g Flour, 5ml Water.

This data is stored as a JSON object in our MySQL database.

### **6.4.2 The 'Multiplier' Logic**

When a Retailer places an order for 100 units of Biscuits, the system automatically performs a calculation:

100 units x 5g Sugar = 500g Sugar needed.

100 units x 10g Flour = 1000g Flour needed.

This requirement is instantly generated and shown in the Manager's Dashboard.

### **6.4.3 Automatic Material Request**

If the Manager clicks 'Instruct Supplier,' the system doesn't just send a vague message. It sends the Specific BoM List to the Supplier's dashboard. This ensures the Supplier knows exactly what ingredients to ship to the Manufacturer. This automation reduces waste, prevents human mathematical errors, and ensures the Manufacturer has exactly what is needed to start production.



## **6.5 Decentralized Communication: Manufacturer & Supplier**

Normally, Manufacturers and Suppliers communicate via phone calls, emails, or WhatsApp. These methods are ‘Unstructured,’ meaning they are hard to track and can be easily deleted or ignored. Our system moves this communication onto the Blockchain Ledger.

### **6.5.1 The Digital ‘Handshake’**

We have implemented a ‘Request-Response’ loop between the Manufacturer and the Supplier:

1. Request (Status 2): The Manager/Manufacturer requests materials on-chain. The Supplier sees an ‘Incoming Request’ alert.
2. Shipment (Status 4): The Supplier ships the materials and clicks ‘Ship & Verify.’ This creates a blockchain transaction.
3. Receipt (Status 5): The Manufacturer receives the goods and clicks ‘Start Manufacturing.’

### **6.5.2 Why this is ‘Decentralized Communication’**

Because this communication happens via the Smart Contract, it is Self-Documenting. There is no need to send a ‘confirmation email.’ The act of clicking the button on the dashboard is the confirmation.

Non-Repudiation: A Supplier cannot later claim ‘I never received the request,’ because the request is visible to everyone on the Sepolia network.

Efficiency: The Manufacturer doesn't need to check their inbox; they simply look at their ‘Production Queue,’ which updates automatically when the Supplier ships the materials.

## **6.6 Automated Inventory Audit via the Blockchain Watcher**

The most technically advanced feature of the system is the Blockchain Watcher Service (watcher.js). This feature solves the problem of ‘Data Silos’ by ensuring the local database always matches the blockchain.

### **6.6.1 Hands-Free Inventory Updates**

In traditional systems, a Warehouse worker must manually type ‘Received 100 units’ into a computer. In our system, this is automated:

The Listener: The Watcher service runs on the Node.js backend, constantly listening for the Status Updated event.

The Warehouse Rule: When the Watcher detects Status 6 (Sent to Warehouse), it automatically runs an `inventory.update()` command to increase the stock level in MySQL.

The Distributor Rule: When it detects Status 8 (Sent to Distributor), it automatically subtracts the stock from the Warehouse and adds it to the Distributor’s inventory.

### **6.6.2 Eliminating Human Error**

Since the computer (The Watcher) handles the math, there are no ‘Typos.’ If 45 units were manufactured on the blockchain, exactly 45 units are added to the database.

Live Stock Levels: Because the Watcher works in real-time, the Manager can look at the ‘Inventory Audit’ tab and see exactly how many units are sitting in the warehouse at that very second.

Alerting System: If the Watcher detects that the stock has dropped below 5 units, the system triggers a ‘Low Stock’ alert (a Red Triangle icon) on the Manager's dashboard, prompting them to start a restock.

### **6.6.3 Data Consistency**

By using the Watcher, we ensure ‘Data Integrity.’ The blockchain acts as the ‘Master Record,’ and the MySQL database acts as the ‘Local Copy.’ The Watcher ensures that the

local copy is never different from the master record, providing a 100% accurate audit trail for the company.

#### 6.6.4 BILL:-

Document Preview

Print Bill

Download PDF

Close

 **SUPPLYCHAIN**  
VERIFIED LEDGER

INVOICE  
ORD: ORD-03XTNT

FROM (DISTRIBUTOR)

BILL TO (RETAILER)

**SONBHADRA DISTRIBUTOR**  
01, Churk Road, Near Police line,  
Churk, Sonbhadra  
555-666-8881

**FOOD CORNER STORE**  
05, Robeertsganj Varansi Road, Near  
Bypass,Robertsganj, S0nbhadra,  
211312  
777-888

| PRODUCT DESCRIPTION | UNIT PRICE | QTY | AMOUNT |
|---------------------|------------|-----|--------|
| <b>BISUCUIT 35G</b> | ₹14        | 145 | ₹2,030 |

GRAND TOTAL  
**₹2,030**

 **BLOCKCHAIN AUTHENTICATED**  
0x81a4E8E3F3255f8948889858d0184c457a497382  
Generated on: 5/4/2026, 1:45:48 AM

AUTHORIZED SIGNATURE

Figure: 6.6.4: bill

## **CHAPTER 7**

### **SWOT Analysis**

I performed a SWOT analysis to assess the strengths, weaknesses, opportunities, and threats.

#### **7.1 Strengths**

- Improved security and tamper-proof record keeping, making it more difficult for fraud or errors to occur.
- Improved traceability, allowing for real-time monitoring and quick identification of issues or errors in the supply chain.
- Increased transparency and accountability, leading to greater trust in the supply chain and its participants.
- Potential for cost savings through increased efficiency and automation of processes.

#### **7.2 Weaknesses**

- Lack of standardization and regulation in the field, making it difficult for companies to adopt the technology and comply with existing regulations.
- High costs associated with implementing and maintaining blockchain technology.
- Limited understanding and expertise in blockchain technology among supply chain management professionals.
- Dependence on network participants' adoption and willingness to integrate the technology.

#### **7.3 Opportunities**

- The ability to create new business models and revenue streams through the use of smart contracts and other blockchain-based applications.
- The potential for increased collaboration and information sharing among supply chain partners through the use of a shared blockchain platform
- The ability to improve supply chain visibility and responsiveness to market changes.
- The potential for better tracking of goods and products through the entire supply chain, from origin to end consumer.

## **7.4 Threats**

- Lack of trust and understanding of blockchain technology among consumers and regulatory authorities.
- The potential for hacking or other security breaches that could compromise the integrity of the blockchain.
- The potential for increased competition from companies that have already adopted blockchain technology in their supply chain.
- Dependence on constantly evolving technology and the need for regular updates and maintenance.

## CHAPTER 8

### CONCLUSION AND FUTURE SCOPE

#### 8.1 Conclusion

The implementation of the Blockchain-Based Supply Chain Management System demonstrates a significant shift from centralized, ‘trust-based’ logistics to decentralized, ‘verification-based’ operations. By utilizing Ethereum smart contracts and a modern React frontend, this project has successfully addressed the primary challenges of traditional SCM: transparency, record tampering, and information silos

Key achievements of this study include.

**Decentralized Trust:** The system removed the need for a central authority to verify transactions, instead relying on cryptographic proof provided by the SupplyChain.sol contract.

**Operational Integrity:** The use of a strict state-machine logic prevented illegal state transitions, ensuring that Every participant, from the raw material supplier to the final retailer, has access to an immutable audit trail, ensuring the authenticity and provenance of goods.the physical movement of goods is accurately and chronologically mirrored on the digital ledger.

In conclusion, while blockchain technology is still evolving, this project proves that it offers a viable and superior alternative for multi-stakeholder collaborative environments where data integrity is paramount.

#### 8.2 Limitation

Despite the successful implementation, several limitations were identified during the development and testing phases:

1. **Gas Costs (Scalability):** Every write operation on the Ethereum network requires ‘Gas’. In a high-volume supply chain, these transaction fees can become prohibitive.
2. **The ‘Oracle’ Problem:** The blockchain is highly secure once data is entered, but the system still relies on humans (Suppliers/Manufacturers) to enter accurate data about the physical world.

3. Privacy vs. Transparency: Public blockchains expose transaction volumes and participant addresses. While names are hidden, the flow of goods is visible to all, which may be a concern for highly competitive businesses.
4. Network Dependency: The system requires all participants to have active internet access and a managed digital wallet (like MetaMask), which may be a barrier in developing regions.

### **8.3 Future Scope**

The current system serves as a robust foundation that can be expanded in several directions:

1. Internet of Things (IoT) Integration: Integrating hardware sensors (GPS, Temperature, Humidity) that write data directly to the blockchain via 'Oracles' would remove the need for manual status updates and prevent human error.
2. Tokenized Payments: Integrating stablecoins (like USDT or USDC) would allow for automated, instant payments. For example, a smart contract could be programmed to release funds to the Manufacturer automatically the moment the Distributor signs for the product.
3. Zero-Knowledge Proofs (ZKPs): Implementing ZKP technology would allow companies to prove they have fulfilled an order without revealing the sensitive pricing or quantity data to the entire network.
4. Cross-Chain Interoperability: Future versions could utilize Layer-2 scaling solutions (like Polygon or Arbitrum) to reduce gas costs while maintaining the security of the Ethereum Mainnet.

## References

- [1] MDPI. (2024). Improving Supply Chain Management Processes Using Smart Contracts in the Ethereum Network Written in Solidity. Applied Sciences <https://doi.org/10.3076/3417/14/11/4738>
- [2] ResearchGate. (2024). Blockchain Technology for Global Supply Chain Management: A Survey of Applications, Challenges, Opportunities & Implications. IEEE Access,
- [3] ITM Web of Conferences. (2025). Blockchain-Based Supply Chain Management Ensuring Transparency and Traceability in Logistics Operations. ICSICE-2025, <https://doi.org/10.1051/itmconf/20257602003>
- [4] Shirkande, S. T., et al. (2025). Blockchain in Supply Chain Management: A Survey of Emerging Trends and Next-Generation Solutions. International Journal of Electrical, Electronics and Computer Systems, <https://doi.org/10.65521/ijeecs.v13i2.11>
- [5] Thakur, C. K., & Kumar, P. (2025). Blockchain in Enhancing Supply Chain Transparency. International Journal for Research in Applied Science and Engineering Technology (IJRASET), <https://doi.org/10.22214/ijraset.2025>
- [6] Marar, H. W., & Marar, R. W. (2026). Hybrid Blockchain: Combining Distributed Ledgers with Centralized Servers for Scalable Business Models. ResearchGate.
- [7] MDPI. (2026). Research on an On-Chain and Off-Chain Collaborative Storage Method Based on Blockchain and IPFS. Future Internet, <https://doi.org/10.3390/fi18020092>
- [8] Immidi, K. (2024). Blockchain Revolution in Supply Chain: A Global Study of 150+ Implementations Across 25 Countries. International Journal of Engineering Research & Technology (IJERT).



- [9] Jurnal UAI. (2026). Design and Implementation of a Blockchain-based Shipment Tracking Proof-of-Concept for Academic Application. Universitas Al-Azhar Indonesia, PSN-2026.
- [10] MDPI. (2025). Unlocking Blockchain's Potential in Supply Chain Management: A Review of Challenges, Applications, and Emerging Solutions. Applied System Innovation, <https://doi.org/10.3390/asi5030034>
- [11] Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. [The foundational paper on blockchain technology].
- [12] Buterin, V. (2013). A Next-Generation Smart Contract and Decentralized Application Platform. [The whitepaper introducing Ethereum and programmable logic].
- [13] Christidis, K., & Devetsikiotis, M. (2016). Blockchains and Smart Contracts for the Internet of Things. IEEE Access, 4, 2292-2303.
- [14] Buterin, V. (2013). A Next-Generation Smart Contract and Decentralized Application Platform. [The whitepaper introducing Ethereum and programmable logic].
- [15] Christidis, K., & Devetsikiotis, M. (2016). Blockchains and Smart Contracts for the Internet of Things. IEEE Access, 4, 2292-2303.
- [16] Francisco, K., & Swanson, D. (2018). The Supply Chain Has No Clothes: Technology Adoption of Blockchain for Supply Chain Transparency. Logistics.
- [17] Kshetri, N. (2018). Blockchain's roles in meeting key supply chain management objectives. International Journal of Information Management.
- [18] Saberi, S., et al. (2019). Blockchain technology and its relationships to sustainable supply chain management. International Journal of Production Research,

- [19] Van Hoek, R. (2019). Exploring the value of blockchain in the supply chain. Journal of Business Logistics.
- [20] Van Hoek, R. (2019). Exploring the value of blockchain in the supply chain. Journal of Business Logistics,
- [21] Solidity Documentation. Solidity v0.8.20. Available at: <https://docs.soliditylang.org/>
- [22] Hardhat Documentation. Ethereum Development Environment. Available at: <https://hardhat.org/docs>
- [23] React Documentation. Components and Hooks. Available at: <https://react.dev/>